



Anurag

@anurag_gharat

https://x.com/anurag_gharat/status/2102227321256337439

A complete guide to all the building blocks in System Design.

System Design Essentials - 1

4

4

17

8

This article is part of a 3-part series on the essential concepts you will use when designing a software system. Once you understand each one clearly, you can combine them to solve almost any problem.

Every topic here is a core building block in system design. Each one has a short explanation, a simple diagram, the problem it solves, and the trade-off behind it.

Read the article end to end and understand each block. After that, system design problems get much easier, whether you are starting from scratch or preparing for interviews.

1. Client-Server Architecture

Client-Server Architecture

Client-Server Architecture is the foundation for almost every system. A client sends requests, and a server processes them while talking to a database. When the client connects straight to a database, it's called 2-tier. Once you add a dedicated application server between the client and the database, it becomes 3-tier, the setup most modern apps actually use. Concepts like caching and scaling sit on top of this basic client-server relationship.

2. Vertical Scaling vs Horizontal Scaling

Scaling a system means improving the capacity of the system to handle more load without breaking. Systems can experience load in the form of more requests, more database access, etc. There are two ways to scale a system

Vertical Scaling vs Horizontal Scaling

Vertical Scaling

In this type, we add more power or improve specs on our existing system. This means

Adding more RAM

Adding more Compute Power

Adding more Storage

Vertical Scaling is great in the initial days, but it soon reaches a ceiling. You can't scale a system beyond a point due to hardware limits.

Horizontal Scaling

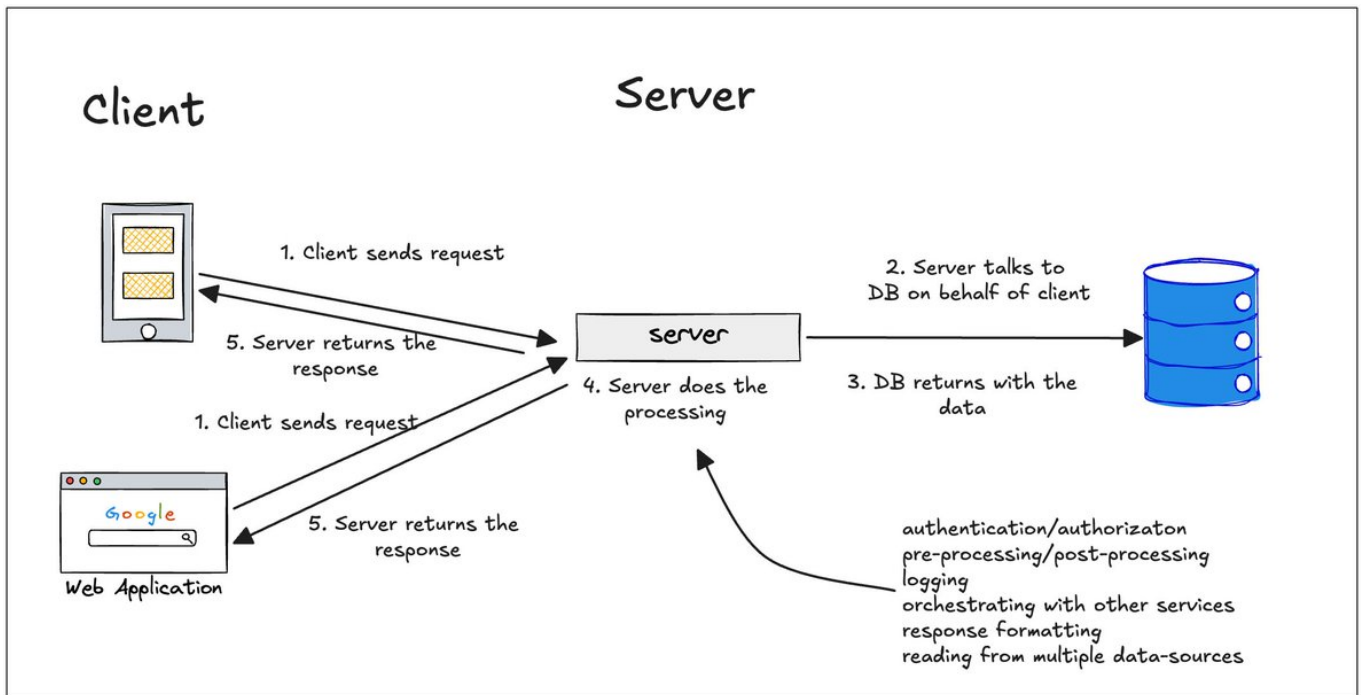
In this type, we add more machines instead of improving the existing machine. This is also called the fan-out pattern. More machines work together in accomplishing a task.

3. Database

Every application needs a place to store data. For an E-Commerce application, we will need a storage solution to store details about users, products, orders, and messages. For a Chat application, we will need to store users, groups, and their messages. This is what a Database is for.

A Database is an organised collection of data that can be stored, retrieved, updated, and deleted reliably even after the server restarts or crashes. A Server does not store data; it is only responsible to read/update/delete data from a database. A Database supports four types of operations:

Create, Read, Update, Delete. Also called CRUD operations.



Types of Databases

Relational Databases

Non-Relational Databases

Graph Databases

Object-Oriented Databases

Hierarchical Databases

Vector Databases

4. ACID vs BASE

Databases use different models to manage transactions, consistency, availability, and scalability. ACID and BASE are two approaches used in database systems, each designed for different application requirements.

ACID Properties vs Base Properties

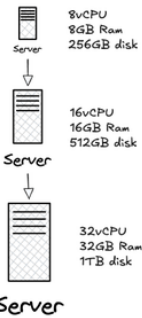
ACID (SQL Databases)

ACID stands for Atomicity, Consistency, Isolation, and Fault Tolerance.

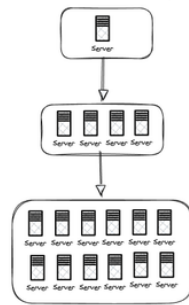
Atomicity - A transaction either fully completes or doesn't happen at all. No partial updates.

Vertical Scaling vs Horizontal Scaling

Vertical Scaling



Horizontal Scaling



- How
- Limit
- Downtime
- Complexity
- Cost

Vertical Scaling

- Bigger machine
- Hardware ceiling
- needs a restart
- simple
- expensive after a limit

Horizontal Scaling

- More machines
- unlimited
- more servers with no downtime
- need load balancers and state handling
- cheaper

For example, in a money transfer request from account A to B.

Step 1: Deduct money from Account A

Step 2: Credit money to Account B

If step 2 fails, step 1 should be rolled back.

Consistency - A transaction takes the database from one valid state to another, never violating defined rules.

Isolation - Transactions running at the same time don't interfere with each other.

Durability - Once a transaction is committed, it stays saved, even if the system crashes right after.

BASE Properties (NoSQL databases)

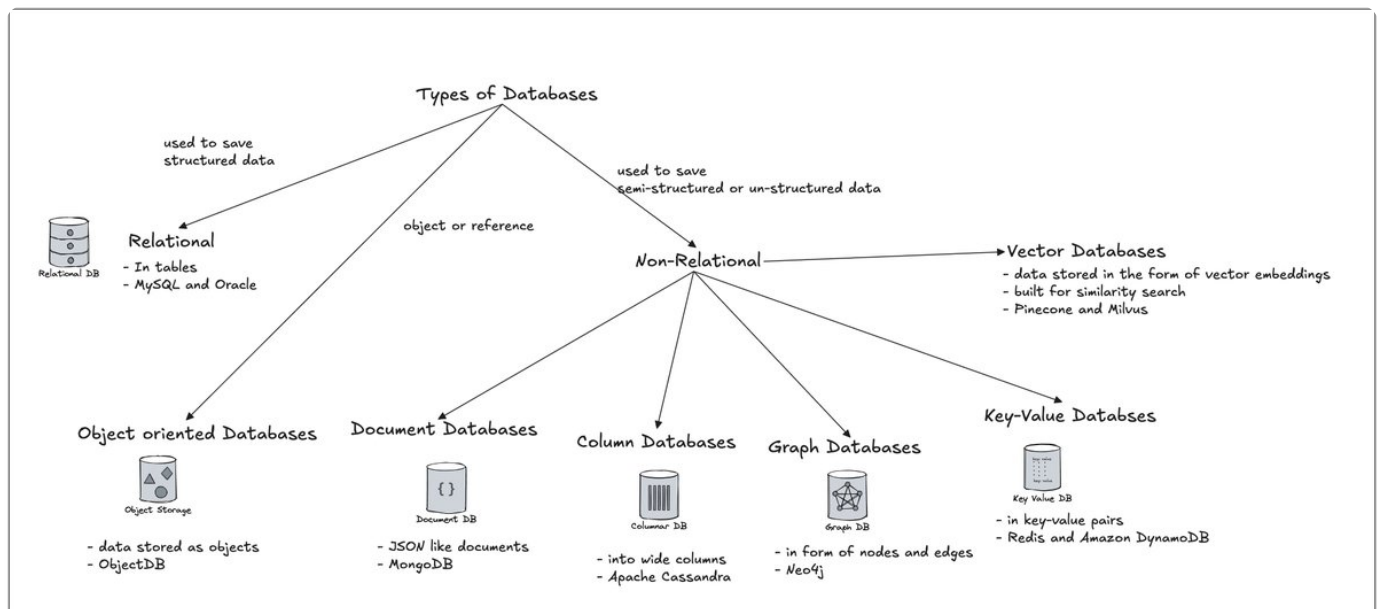
BASE stands for Basically Available, Soft State, and Eventually Consistent. BASE properties prioritize availability over consistency and allow temporary inconsistent data.

Basically Available - The system is available at all times for reads and writes

Soft State - State gets changed without any input due to sync action

Eventually Consistent - Data gets consistent over time and not instantly.

5. CAP Theorem



CAP Theorem

CAP Theorem states that in a distributed system, you can only guarantee two out of these three at the same time.

Consistency: Every read gets the most recent write, no matter which node handles it.

Availability: Every request gets a response, even if some nodes are down.

Partition Tolerance: The system keeps working even if network communication between nodes breaks.

Why not all three?

Network partitions will happen eventually. Once nodes can't talk to each other, you're forced to choose.

CP systems - Choose Consistency over Availability during a partition.

AP systems - Choose Availability over Consistency during a partition.

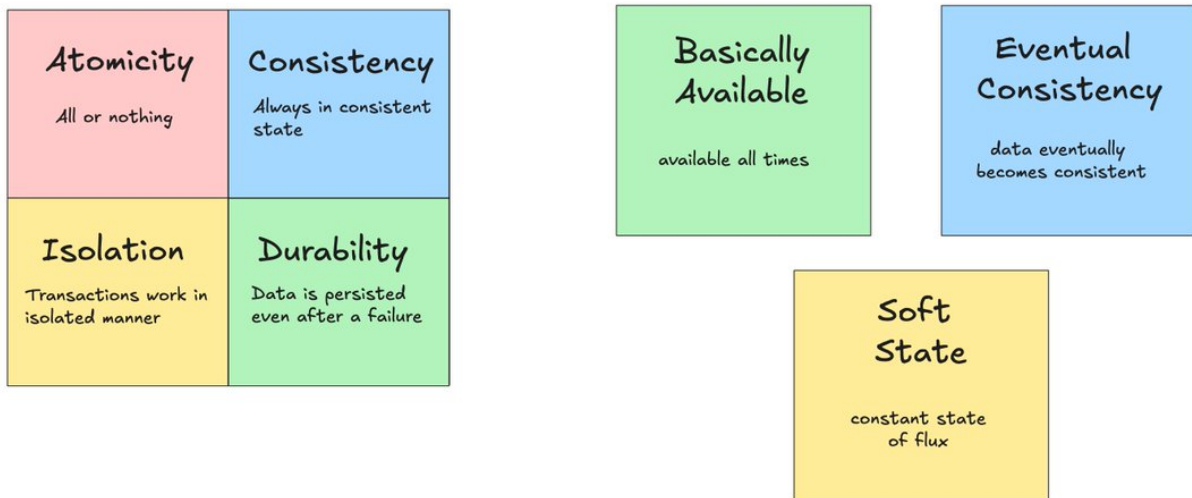
CA systems - Choose Consistency and Availability and give up on tolerance to partition.

ACID leans CP. Strict correctness, willing to reject requests to stay consistent.

BASE leans AP. Stays available, accepts eventual consistency.

6. Relational Databases

ACID vs BASE



Relational Databases

Relational Databases, also called SQL databases, are a type of database where data is stored in rows and columns, organized into tables. The data follows a strict, predefined schema, so every row in a table has the same structure. Tables are connected to each other through foreign keys, which represent relationships between different entities, like a user and their orders.

This structure lets you run complex queries using joins, combining data across multiple tables in a single query. SQL databases also follow ACID properties, making them reliable for use cases where accuracy matters, like banking and order management.

Common examples of Relational Databases: PostgreSQL, MySQL, and Oracle.

7. Non-Relational Databases

Non-Relational Databases

Non-Relational Databases, also called NoSQL databases, are a type of database where data doesn't need to follow a fixed table structure. Instead, data is stored in flexible formats like documents, key-value pairs, or wide columns, depending on the database type.

Each record can have a different structure, so you're not locked into a strict schema upfront. This flexibility makes NoSQL databases well suited for unstructured or fast-changing data, and for applications that need to scale horizontally across many servers. Most NoSQL databases favor availability and speed over strict consistency, following BASE instead of ACID.

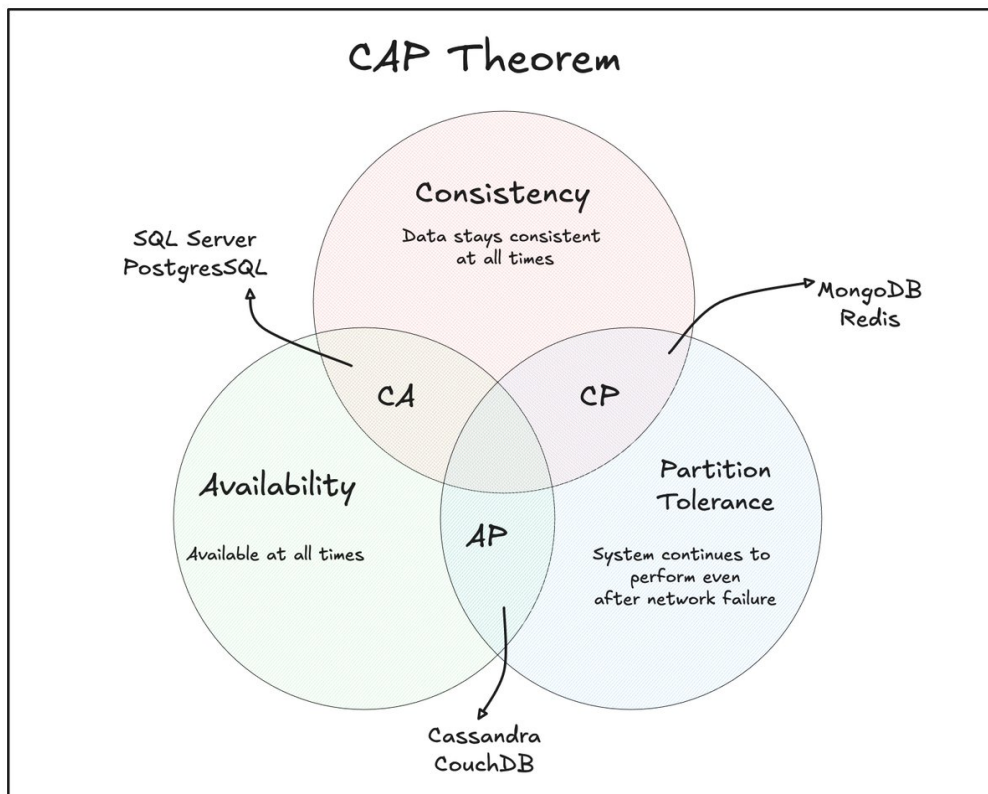
Common examples include MongoDB, Redis, and Cassandra.

7. Database Replication

Database Replication

Creating and maintaining copies/replicas of the same database, kept in sync with each other, is called Database Replication.

The core pattern: Primary-Replica(master-slave)



Primary(Master) - handles all writes

Replicas(Slaves) - handle all reads and continuously sync data with the primary

Most applications are read-heavy, i. e., way more people browsing products than placing orders. By routing reads to replicas, the Primary is freed up to focus only on writes, and you can add more replicas to handle more read traffic.

1 Primary (writes) -> can't scale infinitely

5 Replicas (reads) -> scale read capacity easily by adding more

Types of Replications

Synchronous Replication: The primary waits for the replica to confirm the write command before responding to the server (client for the DB).

Client -> write -> Primary -> write to replica -> confirm to client

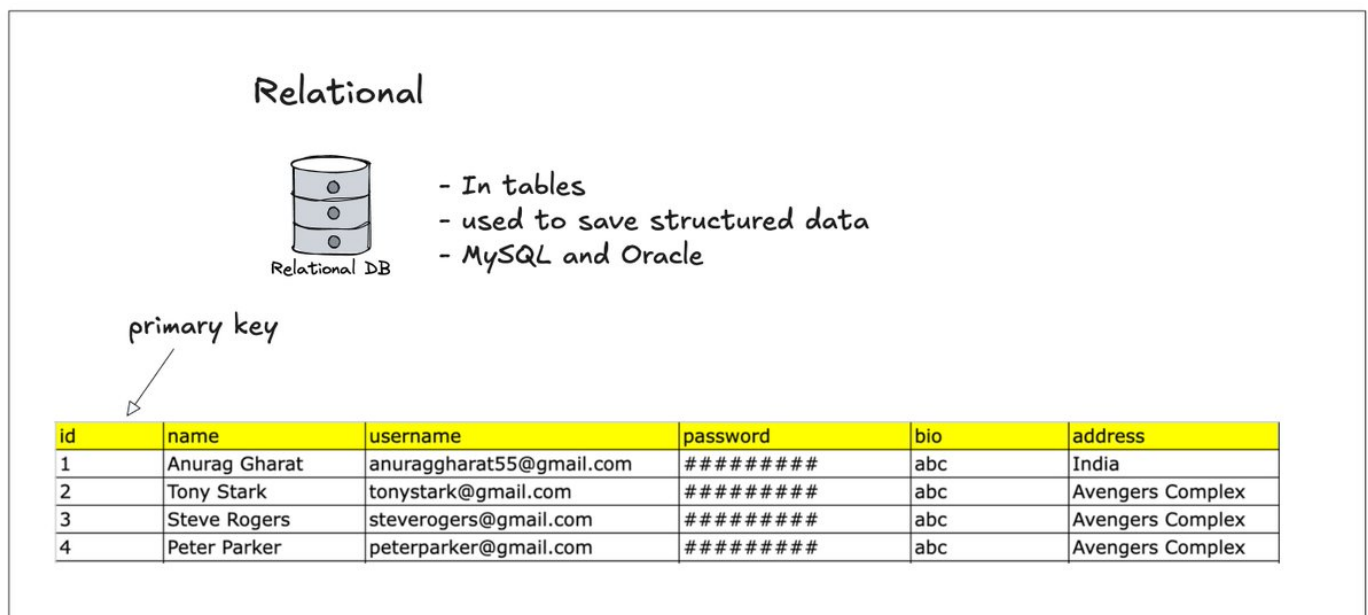
Asynchronous Replication: The primary responds to the client immediately and signals the write to replicas in the background.

Client -> write -> Primary -> respond to Client -> run background write to replica.

8. Database Partitioning

Data Partitioning

Data partitioning is the practice of breaking the main database into separate smaller databases for processing and storage. There are two types of database partitioning.



Vertical Partitioning: In this type, we split the tables by columns. Different columns of the same row live in different tables or stores.

Horizontal Partitioning: In this type, we split a table by rows. Same columns, but different rows live in different partitions.

Benefits of Database Partitioning:

Smaller pieces are easy to manage and scale

Easy and fast lookups

Data that is no longer needed can be moved to a separate database and archived.

9. Database Sharding

Database Sharding

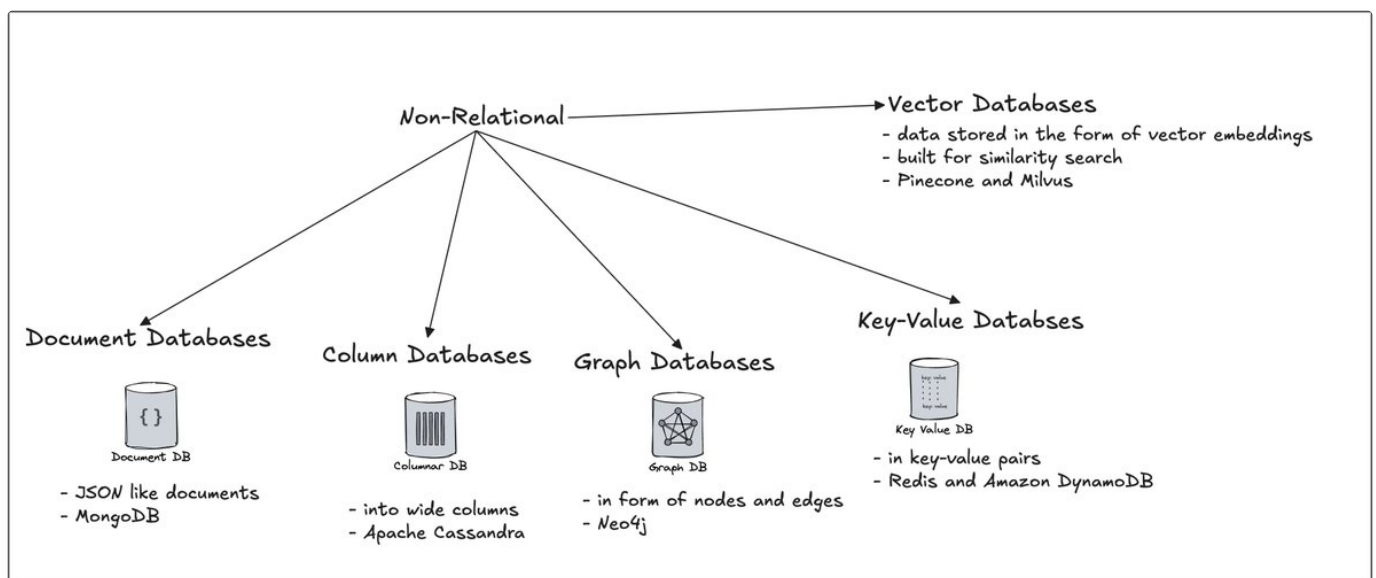
Database Sharding is a way of scaling a database by splitting it into smaller, independent pieces called shards, each holding only a portion of the total data. Instead of one database handling every read and write, the load gets distributed across multiple shards, each running on its own server. Database Sharding is a Horizontal type of Database Partitioning.

A sharding key decides which shard a piece of data belongs to, commonly using range-based, hash-based, or geo-based strategies. This helps when a single database can no longer handle the write load or the total data size, something replication alone can't fix, since every replica still stores the full dataset. The tradeoff is that queries spanning multiple shards become harder, and cross-shard transactions aren't simple to keep atomic.

10. HTTP

HTTP - HyperText Transfer Protocol

HTTP stands for HyperText Transfer Protocol and is a protocol that is used by clients and servers to communicate and transfer data with each other. HTTP contains a set of rules on how communication is established, requests are formatted and sent, and responses are sent. HTTP is a stateless protocol and uses TCP underneath.



Every HTTP request contains:

Method - what action to perform

URL - address of the resource

Headers - Request metadata

Body - Data sent by the client to the server

Every HTTP response contains:

Status Code - result of the request

Headers - metadata of the response

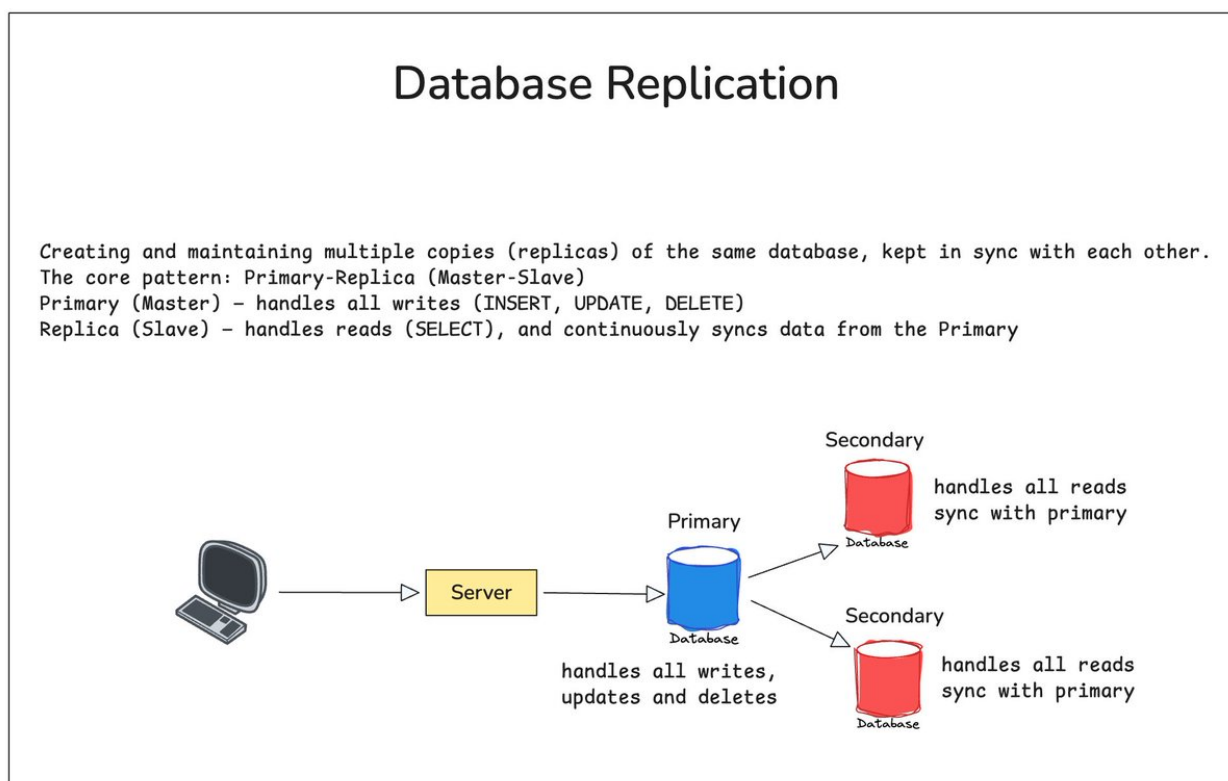
Body - requested data

HTTP methods:

GET - get data

POST - create new data

PUT - update/replace data



PATCH - update data in place

DELETE - delete data

HTTP status codes

200 - OK

201 - Created

301 - Moved

304 - Not Modified

400 - Bad Request

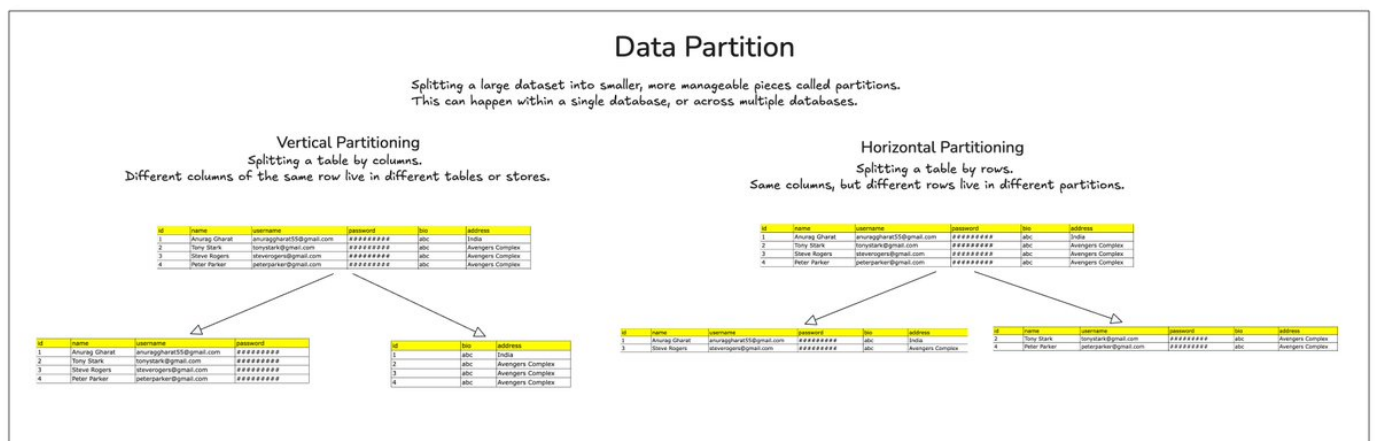
401 - Unauthorized

404 - Not Found

500 - Internal Server Error

503 - Service Unavailable

11. Monolithic Architecture



Monolithic Architecture

Monolithic Architecture is a single unified codebase where all the features of an application reside together and run as a single unit.

One codebase - One build - One Deployment

Monolithic Architecture is further divided into two more types

Traditional Monolithic: An application where all the features, including the User interface, Business Layer, Data access layer, and database, all reside within the same repository. For example: A Java or Spring application

Modular Monolithic / Modern Monolithic: A modern form of the monolith where only the Service layer features remain monolithic, whereas the UI and database are kept separate.

Advantages of Monolithic Architecture:

Easy to set up, develop, test, and deploy

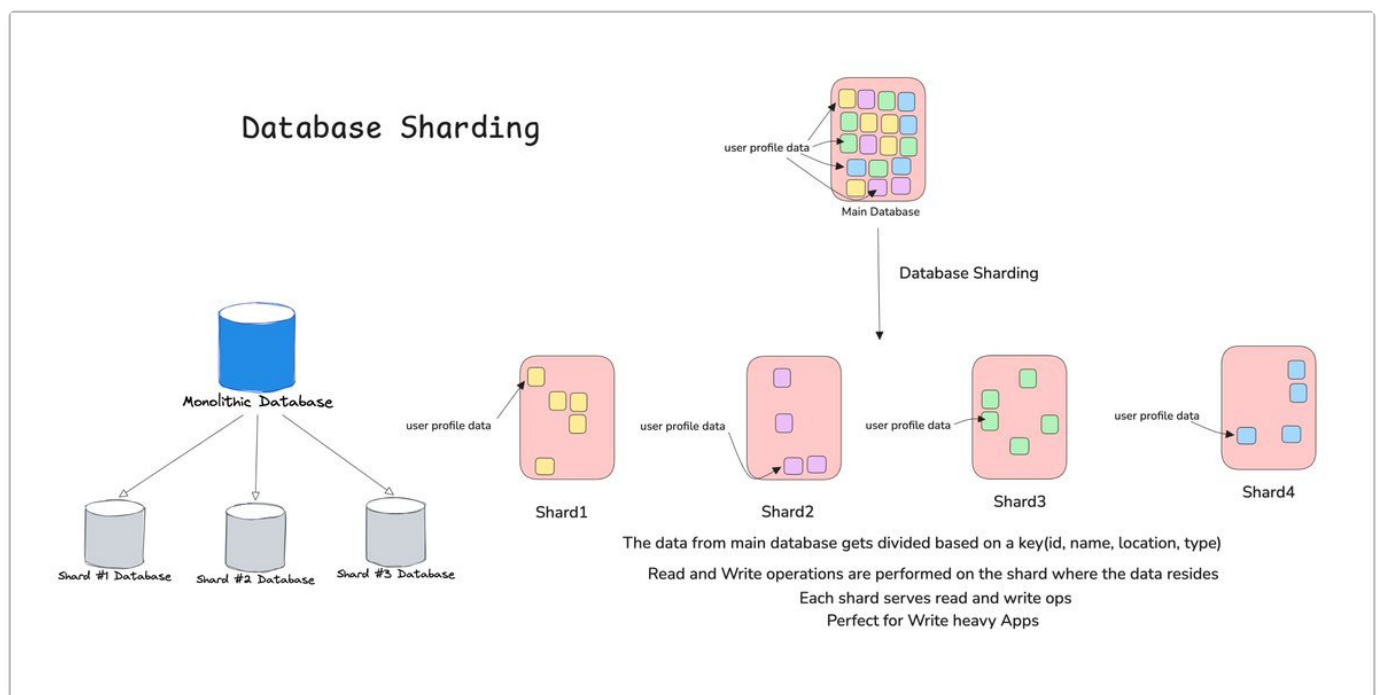
Fast speed of execution

Great for smaller teams and limited scope

Disadvantages of Monolithic Architecture:

Difficult to maintain once the application starts growing

Tight coupling within layers



Difficult for multiple teams to work independently

Lock-in for the tech stack.

12. Microservices Architecture

Microservices Architecture

In a Microservices Architecture, instead of building one large application, the system is divided into multiple small, independent services. Each service is responsible for a single business functionality, maintaining its own codebase, deployment pipeline, and often its own database. This architecture allows applications to be designed as a collection of loosely coupled, interdependent services that can be developed, deployed, and maintained independently.

In an E-Commerce app built with microservices:

User Service handles authentication and profiles

Product Service handles catalog and inventory

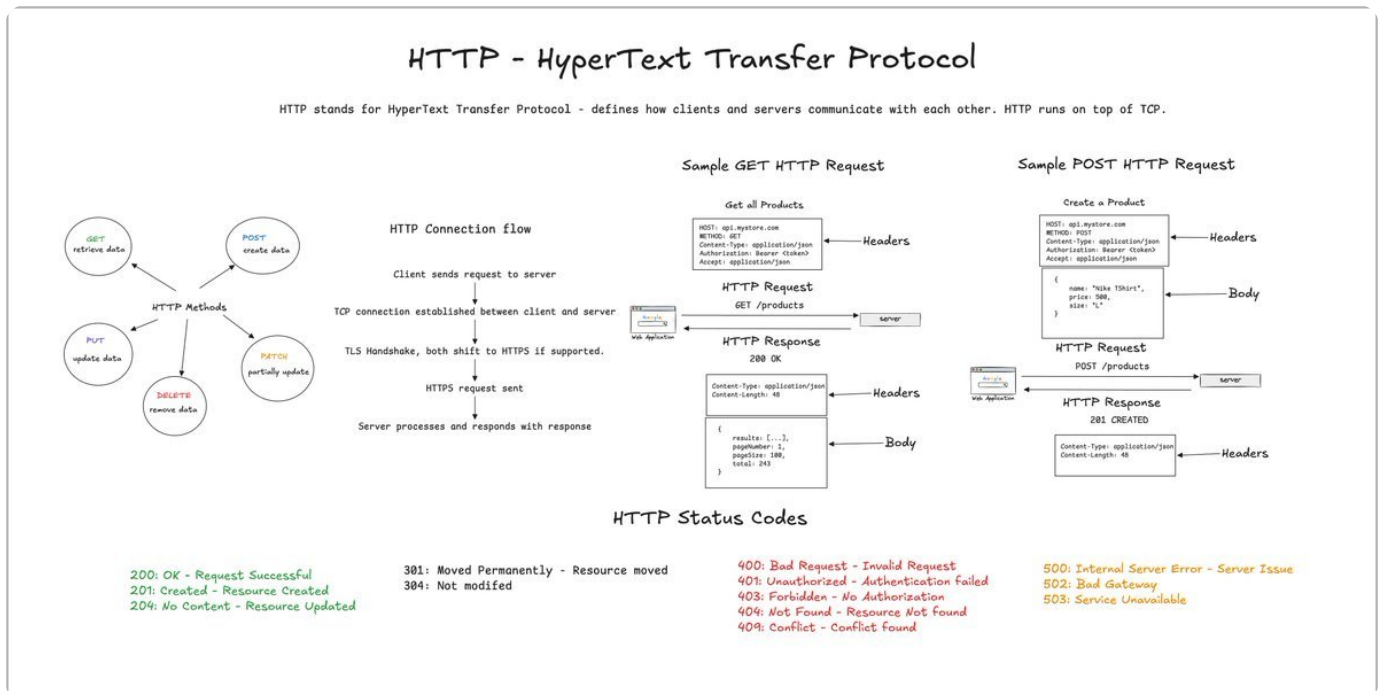
Order Service handles order creation and tracking

Payment Service handles transactions

Advantages of Microservices:

Independent Deployability

Scalability



Fault Isolation and separation of concerns

Technology Flexibility

Faster Development Cycles

Disadvantages of Microservices:

Complex Communication

Data Management

Operational Overhead

Testing Complexity

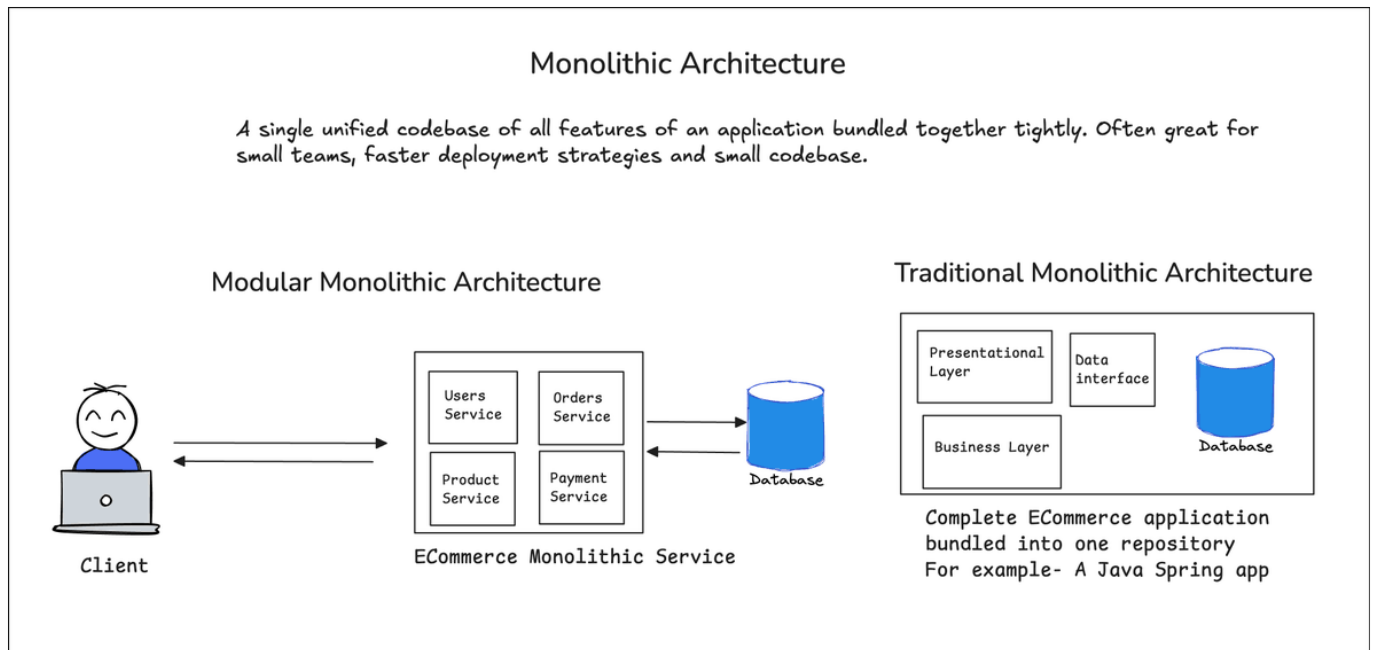
Latency

13. API Gateway

API Gateway

An API gateway sits between the client and server and only exposes the part needed for the client, hiding all the complexity. An API gateway is a single point of entry for all clients of an application. It sits between the clients and microservices.

Client -> API gateway -> Microservices



Responsibilities of an API gateway include:

Handling authentication and authorisation for each request from the client.

Routing the request to the correct microservice

Logging and monitoring

Load balancing

Rate limiting

Caching

Changing protocols (requests can be HTTP, changed to gRPC for service-to-service communication).

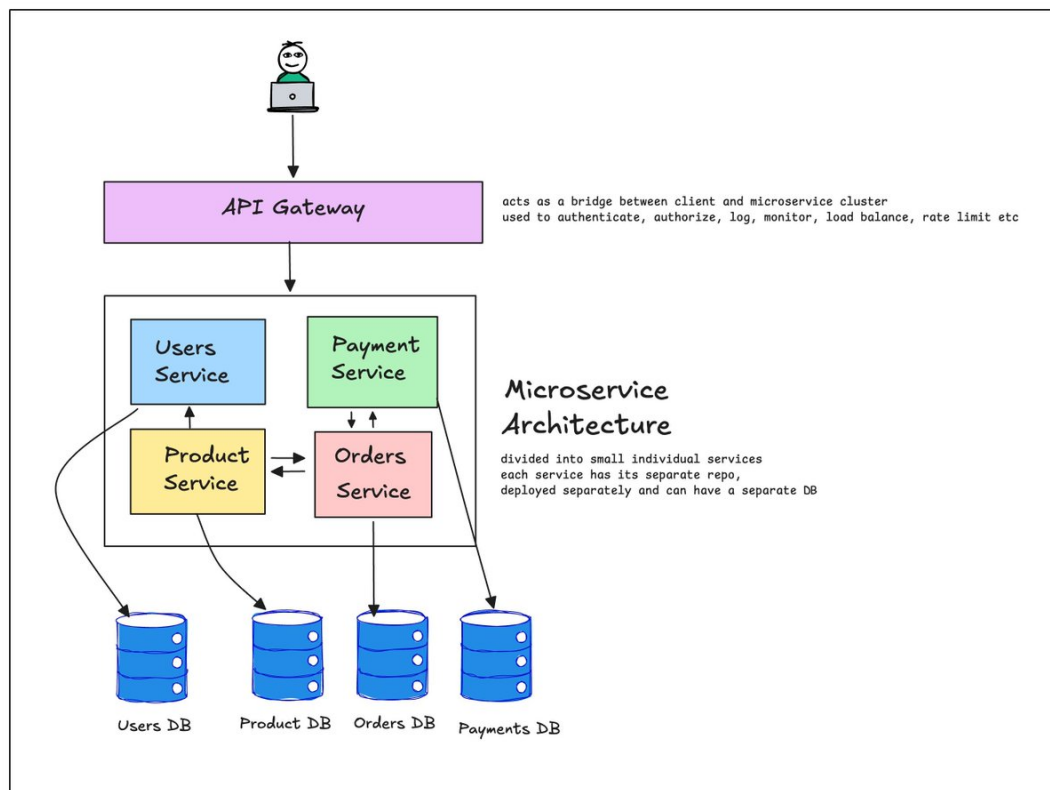
Amazon's API Gateway is one of the most common API gateway services. Other services include NGINX and Kong.

14. REST APIs

REST APIs

REST, which stands for Representational State Transfer, are sets of rules built on top of HTTP for designing an API(Application Programming Interface). REST APIs are like a set of rules and a contract for entities (client and server) to interact with each other predictably.

REST follows six principles



Uniform Interface: Consistent naming, standard interface, consistent structure, consistent use of HTTP methods and status codes across the entire API.

Stateless: No request is saved, and every request must contain all the information that the server needs.

Client-Server: Separation of interfaces and concerns between client and server.

Cacheable: Responses can be explicitly marked as cacheable or non-cacheable and can reuse previously fetched responses

Layered System: Multiple layers can be introduced between the base architecture(client-server). The client cannot see beyond the immediate layer they are interacting with and is not concerned about whether it's called to the end server.

Code-On-Demand: Servers can extend or enhance the client's functionality by sending executable code to the client.

15. SOAP APIs

SOAP, which stands for Simple Object Access Protocol, is a simple protocol for exchanging structured information between systems. It has strict rules on message formats, and communication happens only in

XML. Unlike REST, which is an API style and uses JSON/XML and plain text for communication. SOAP is far more rigid and strict.

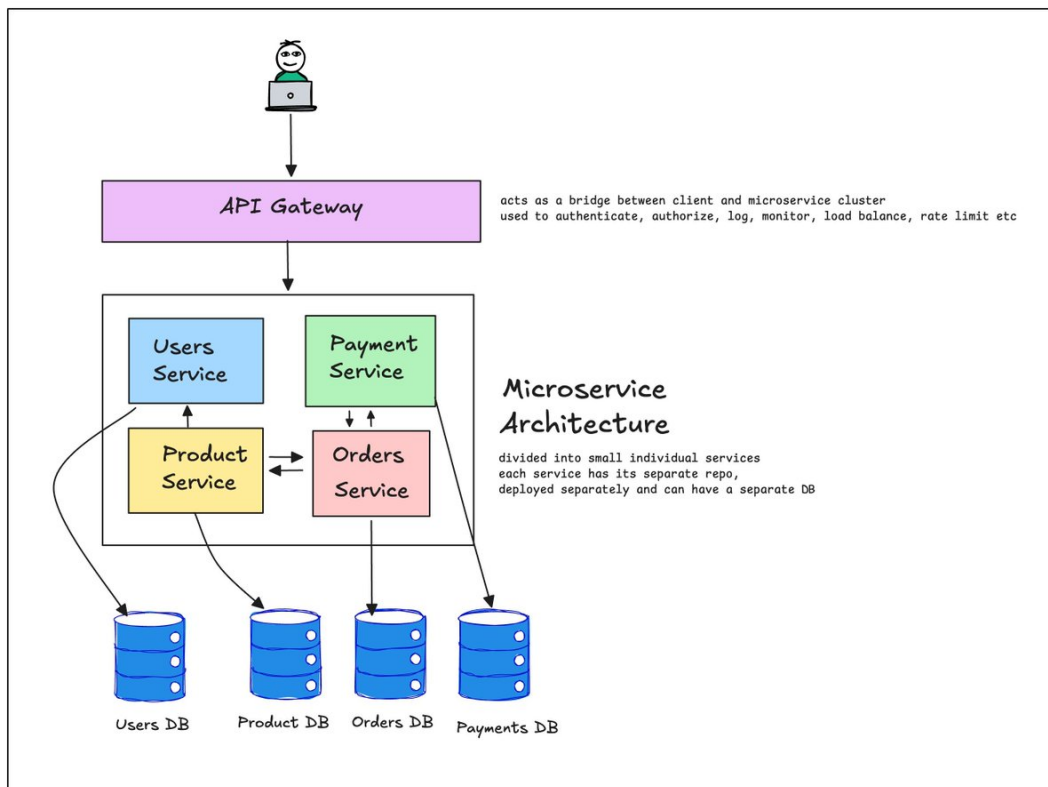
Every SOAP request/response is wrapped in an XML structure called an Envelope. The Envelope header has the metadata, while the body has the actual request/response data alongside the error details.

Example SOAP Request:

xml

<soap:Envelope>

<soap:Body>



<GetProduct>

<ProductId>45</ProductId>

</GetProduct>

</soap:Body>

</soap:Envelope>

16. GraphQL

GraphQL

REST APIs have two problems

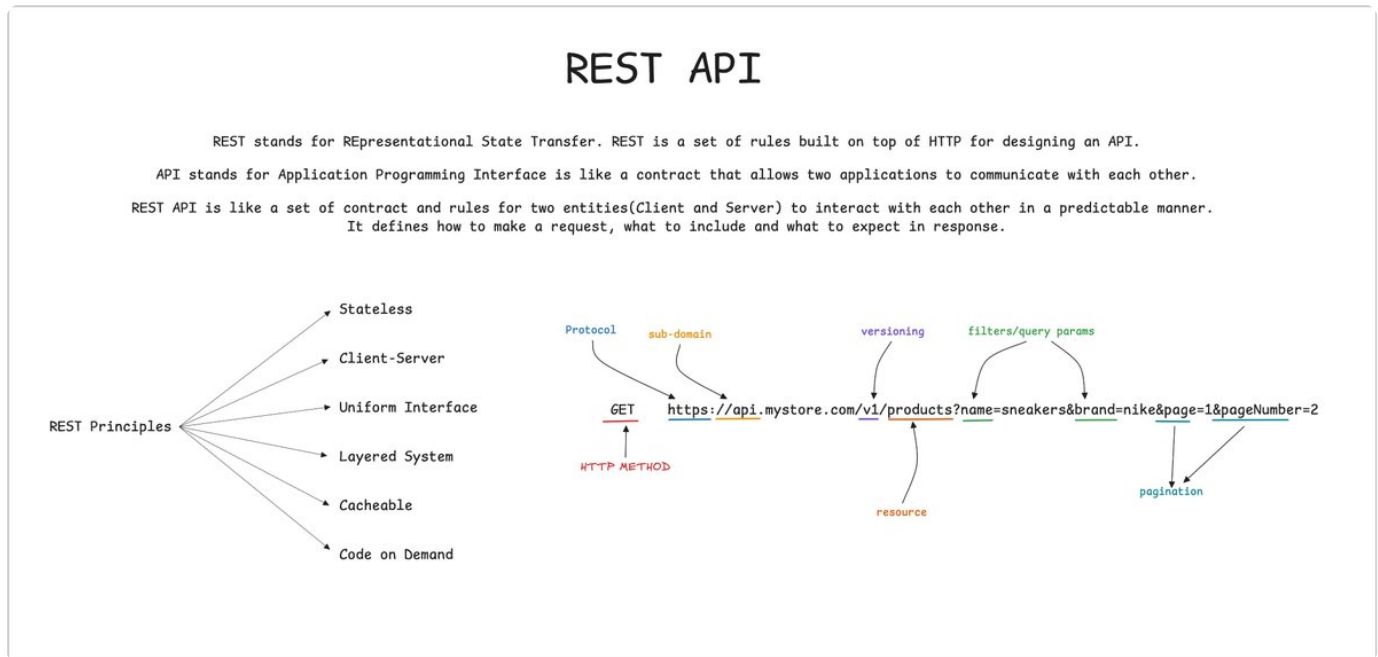
Overfetching: Fetching more data than needed.

Underfetching: Need more than one API call since one API response is not enough.

GraphQL fixes both problems. GraphQL is a query language for APIs where the client specifies exactly what data it needs, in a single request- nothing more, nothing less. Unlike REST (multiple endpoints, fixed responses), GraphQL exposes a single endpoint, and the client shapes the response.

POST /graphql.

That's it. One endpoint for everything.



Example: Fetching a profile page

REST equivalent (2 calls):

text

GET /users/12

GET /users/12/orders?limit=3

Each call returns its own full object -- you get fields you don't need (email, address) and still have to stitch two responses together on the client.

GraphQL (1 call):

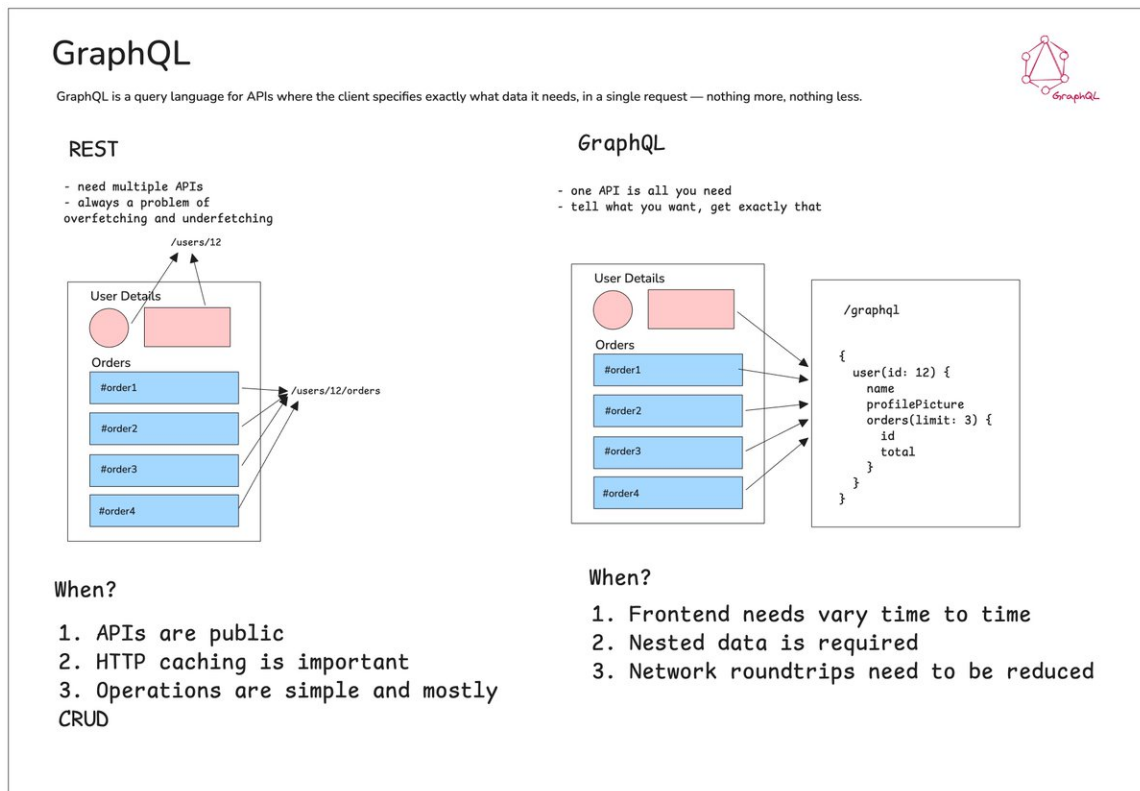
Request (what the client asks for):

javascript

```
{  
  user(id: 12) {
```

name

profilePicture



```
orders(limit: 3) {
```

```
  id
```

```
  total
```

```
}
```

```
}
```

```
}
```

Response (exactly matching the shape requested):

```
text
```

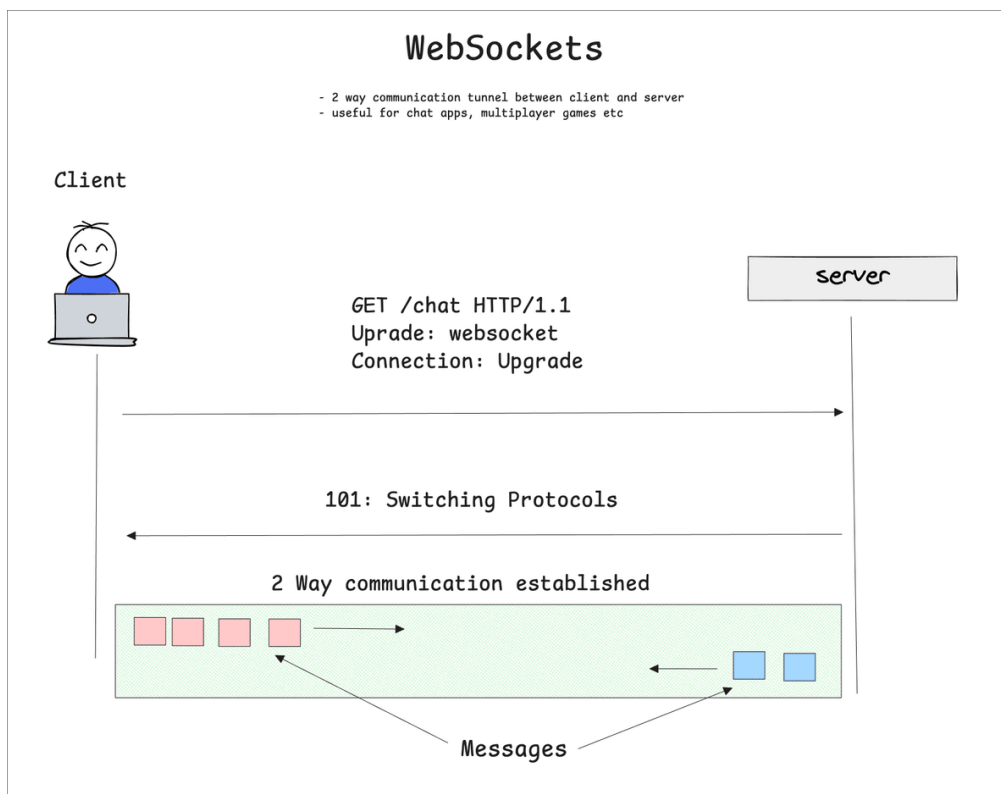
```
{
```

```
  "user": {
```

```
    "name": "Anurag",
```

```
    "profilePicture": "url...",
```

```
    "orders": [
```



```
{ "id": 89, "total": 1200 },
{ "id": 84, "total": 850 },
{ "id": 77, "total": 2100 }
]
}
}
```

One request. Exact fields. No over-fetching, no under-fetching, no stitching responses together.

17. WebSocket

Websocket

A WebSocket is a persistent, two-way connection between a client and a server. Once a connection is established, the client and server can both send messages to each other without repeated connection establishments or handshakes.

A WebSocket connection starts as a regular HTTP request but upgrades to a WebSocket connection once the server responds with a 101 status code. Once the 101 status code is received, the request-response cycle ends. It becomes an open, two-way communication channel.

Real-world examples:

Chat apps

gRPC - Google Remote Procedure Calls

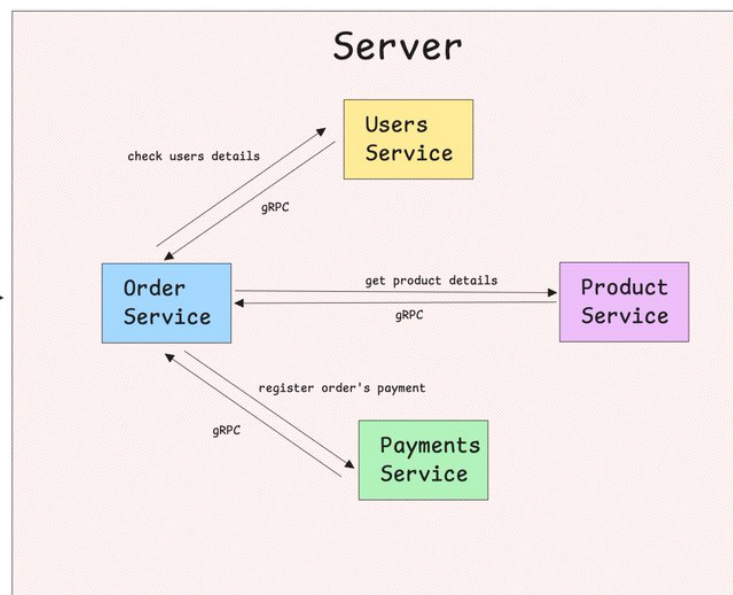
- fast communication framework developed by Google for communication between systems.
- It internally uses Protocol Buffer and HTTP/2.
- Best suited for service to service communication.

Client



places an order
REST - POST /orders

Server



Online games

Sports or Stock apps

Food Delivery apps

Collaborative tools like Google Docs.

18. gRPC

gRPC

gRPC, which stands for Google Remote Procedure Call, is a high-performance framework that lets one server call a method on another server running on a completely different machine as if it's a local function call.

For example, inside an OrderService you can do

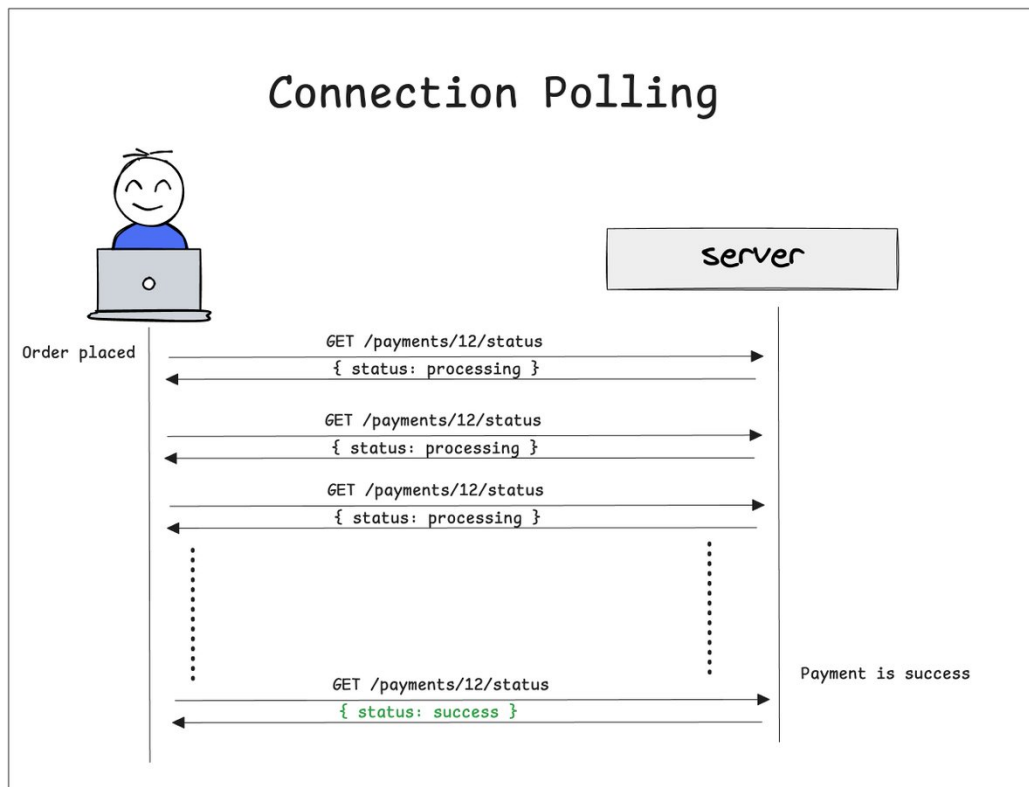
javascript

```
ProductService.getProductDetails(order.productId)
```

```
UserService.getUserDetails(order.userId)
```

This feels like calling a function from the same file, but under the hood, there's a network call to a different server running on a different machine entirely.

How is gRPC so fast?



Uses ProtoBuf, gRPC serialises data into a compact binary format. Smaller payload, faster to parse.

Uses HTTP/2, which is faster than HTTP/1.1 (used by REST)

When to use gRPC?

Server-to-server communication where speed matters

Low-latency requirement systems

Systems with streaming

19. Polling

Web Polling

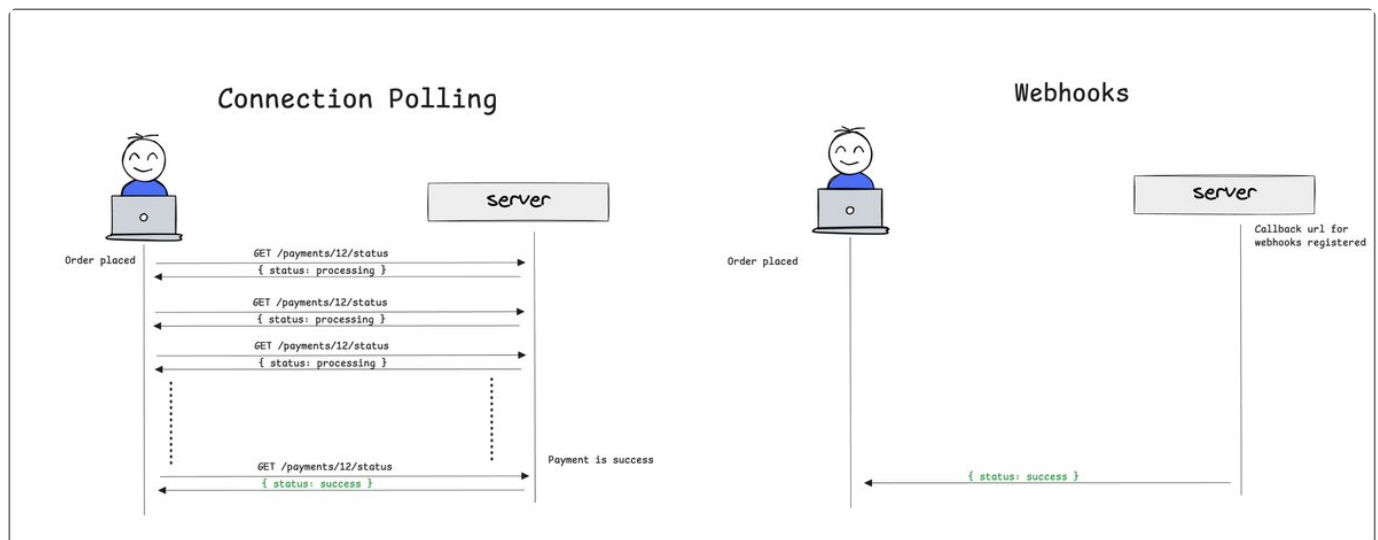
Imagine you are building an E-commerce application checkout page. On this page, the user has to make a payment, and they can use their phone to scan the QR code and pay. Here, the client should auto-update the payment status without the client clicking on refresh. We need the server to notify the client once the payment is successful without the client asking for it explicitly. Without Webhooks, our app would have to ask the PaymentService for the payment status every second.

text

Client -> GET /payments/33/status

Client -> GET /payments/33/status

Client -> GET /payments/33/status



Client -> GET /payments/33/status

This is called Polling. The client repeatedly asks the server if there is any change. The client reacts based on the response provided by the server.

Used in:

Payment Gateways

Live Dashboards

Chat apps

Disadvantages

Inefficient and a waste of resources

The server can get overloaded due to continuous requests

20. Webhooks

WebHooks

Webhooks are a better alternative to Web Polling. A Webhook flips the direction of communication. Instead of the client requesting the server first, the server calls the client automatically the moment an event occurs.

Client -> "Here's my URL, call me when the payment is done" -> Server saves this URL

...time passes, event happens...

Server -> POST `https://yourapp.com/webhook-handler` -> Client.

The client never asks what happened, but the server pushes the update once it happens. REST, GraphQL, and gRPC are all pull-based techniques, i.e., the client pulls for information. Webhooks are push-based, i.e., servers push the events to the clients.

That's it for the first part. Stay tuned for the second and last part, where I will explain topics like Event-Driven Architecture, Load Balancing Algorithms, Rate Limiting Algorithms, Worker Queues, etc.



Anurag

@anurag_gharat

https://x.com/anurag_gharat/status/2104493200870433145

Here's a part 2 of the System Design Essentials Series. Contains 20 more concepts like Load Balancing Algorithms, Event-Driven Architecture, Pub-Sub, Message Queues, etc.

System Design Essentials - 2

Anurag

5

24

33

Knowing the components of a system is not the same as knowing how to put them together.

Part 1 covered the essentials. Part 2 is about the decisions that follow: which architectural style to pick, where to optimize, and what those choices cost you later.

This is the second article in a 3-part series on system design. Part 1 is below:

Anurag

@anurag_gharat

Sep 22

A complete guide to all the building blocks in System Design.

Article

System Design Essentials - 1

This article is part of a 3-part series on the essential concepts you will use when designing a software system. Once you understand each one clearly, you can combine them to solve almost any problem....

23

210

1.4K

114K

21. Event-Driven Architecture

Event-driven Architecture

Event-Driven Architecture (EDA) is a design pattern in which services and applications communicate asynchronously through events. In synchronous communication, the calling service waits for a response from another service before moving forward. This creates tight dependencies between services. If one service starts responding slowly or goes down, the entire flow goes down.

In Event-Driven Architecture, instead of services calling each other directly, a Service triggers an event and doesn't wait for any other service to respond. Other services listen to this event and take necessary actions based on it.

Service A -> Raise an event -> send response back to the user request

Service B -> listen to the event and react

Building Blocks in Event-Driven Architecture

Event: A message about something happened (ORDER_PLACED, USER_CREATED, ORDER_DELIVERED)

Producer: The service that emits the events

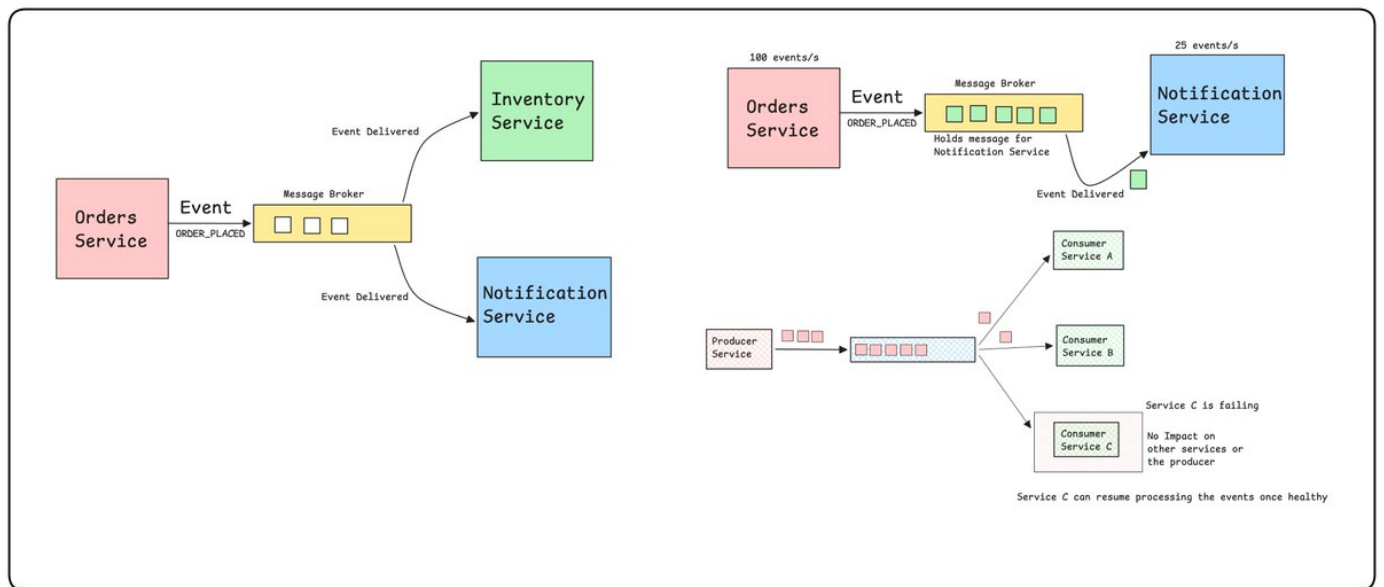
Consumer: A service that listens to events

Event Broker/ Message Queue: A service to hold the messages. Receives the message and delivers it to the consumer

Advantages

Loose Coupling between services

Consumers can scale independently



Asynchronous Processing

Easy to add a new Consumer

22. Serverless Architecture

Serverless Architecture

Monolith and microservices both run on servers you manage. Serverless does not. The name is misleading. Servers still exist. You just don't provision, scale, patch, or shut them down. The provider does that.

You write a function and deploy it. The platform runs it when a request comes in and stops it when the work is done. Serverless is an on-demand server, not the absence of one.

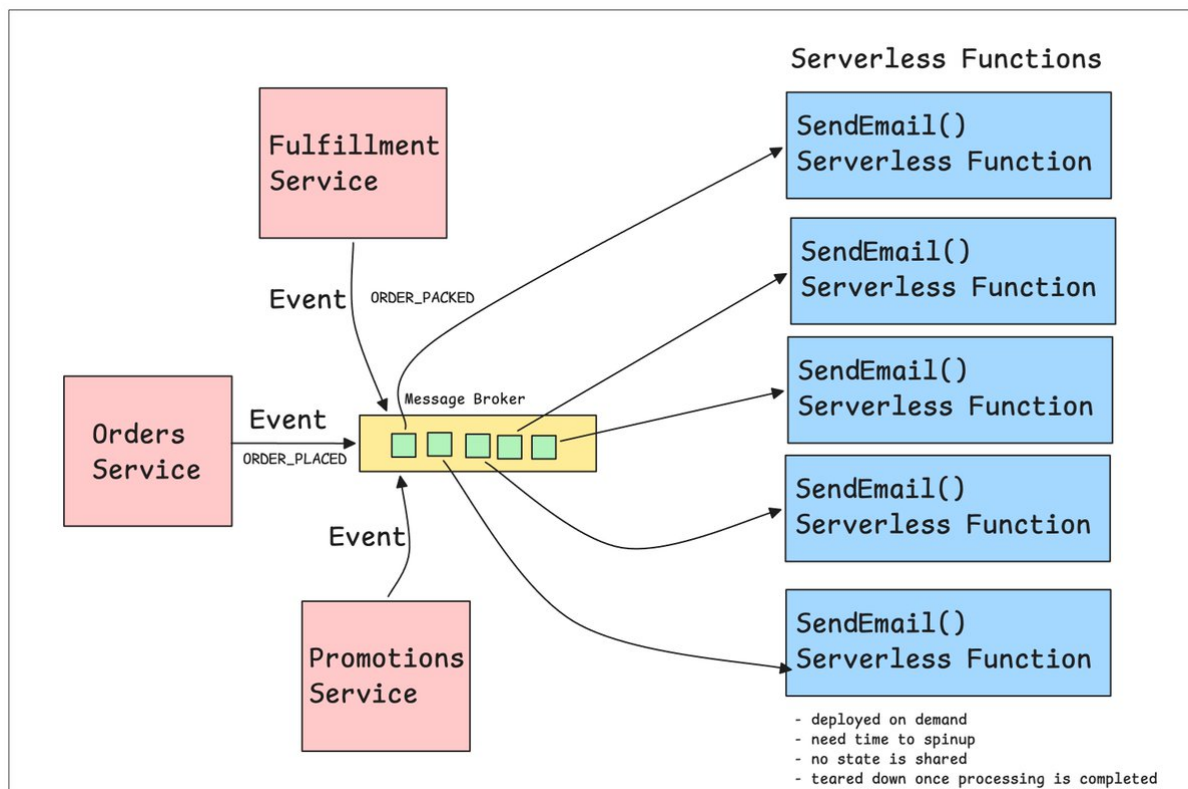
Event-driven: Functions are triggered when a request arrives (Events are triggered).

Auto Scaling: If 1000 requests are received, as many functions will be spawned.

Cold Start: Since a new environment needs to be set up for processing any request, there is a delay in processing, which is called a cold start.

Stateless: Each function is independent and shares no state with any other function.

Pay-per-use Billing: Billed based on your usage.



When Serverless makes sense:

Event-driven tasks

Infrequent or unpredictable traffic

Rapid Development

23. Content Delivery Network - CDN

Content Delivery Network

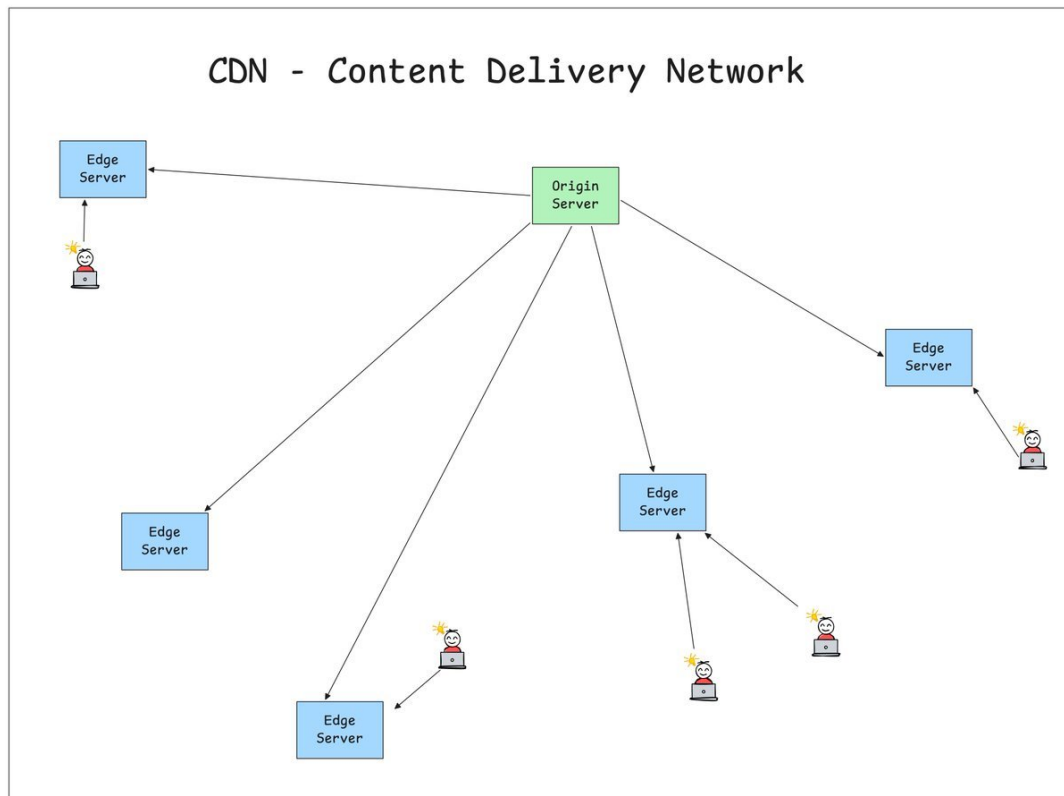
CDN, which stands for Content Delivery Network, is a network of geographically distributed servers that cache and deliver your content from a location physically closer to the user instead of the request travelling back to the origin server.

How does it work?

The origin server is the actual server where all the real content lives.

Content gets cached on Edge Servers all across the world.

Static content like CSS, JS, HTML, and images gets cached on these edge servers and served through them.



Every user request is routed to the nearest Edge server.

In case of a cache hit -> return immediately

In case of a cache miss -> get from the origin server, create a copy on the edge server, and return the response

CDNs are best suited for static or infrequently changing content, not real-time, highly dynamic data like a live bank balance.

24. Message Queues

Message Queues

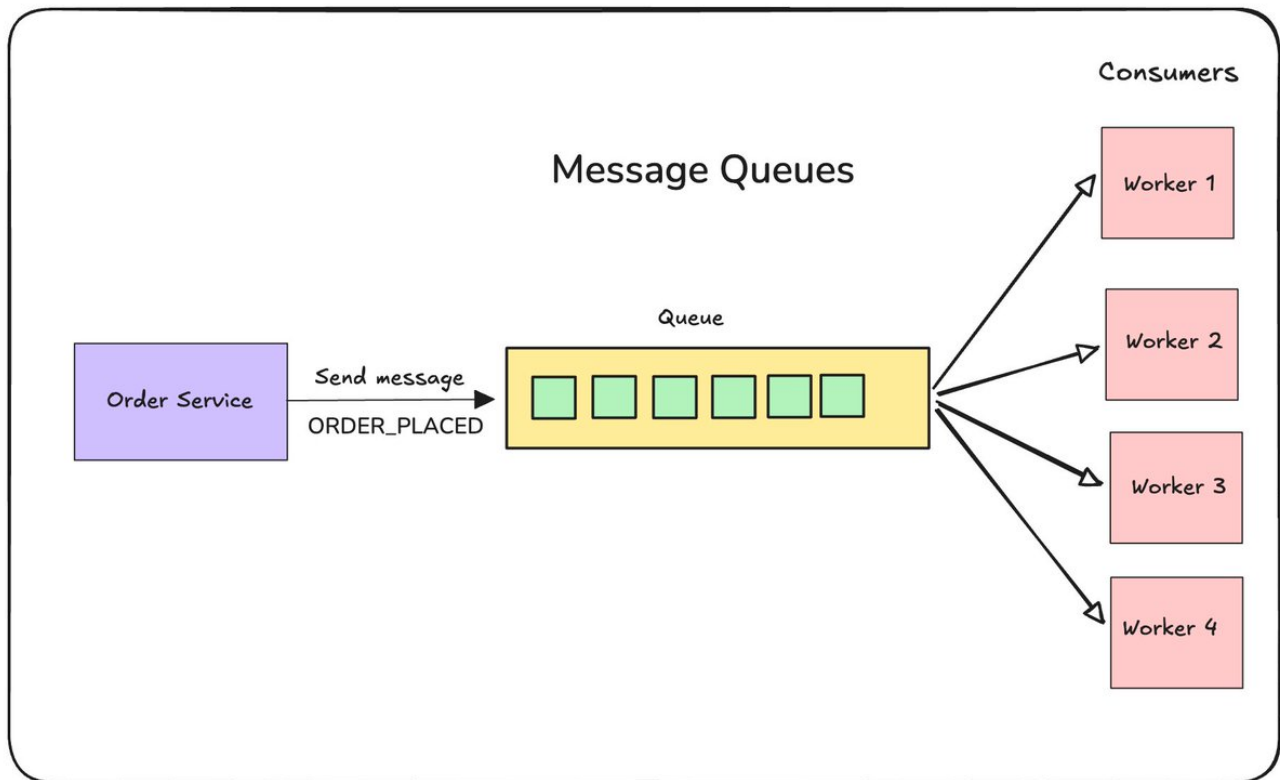
A Message Queue is a buffer between a producer and a consumer that holds messages until they're processed, so the producer doesn't have to wait for the consumer to be ready.

Producer -> pushes message -> Queue -> Consumer picks it up whenever ready

Why can't we call the consumer directly?

In a direct Producer -> Consumer call, the Consumer must be up, fast, and available at all times. If the Consumer is down or recovering, the call will fail, and hence the request fails.

A queue sits in between the Producer and Consumer:



Producer -> Queue -> Consumer

Producer doesn't care if the Consumer is slow or down. It pushes the message to the queue, and its job is done. The consumer can, at its level, access the message and perform its action. If the consumer crashes, messages just wait in the queue and nothing is lost.

Real-world example:

Order Service -> pushes "SendInvoice" to Queue

Order Service -> responds to client immediately

Invoice Worker -> pulls message from Queue -> generates invoice

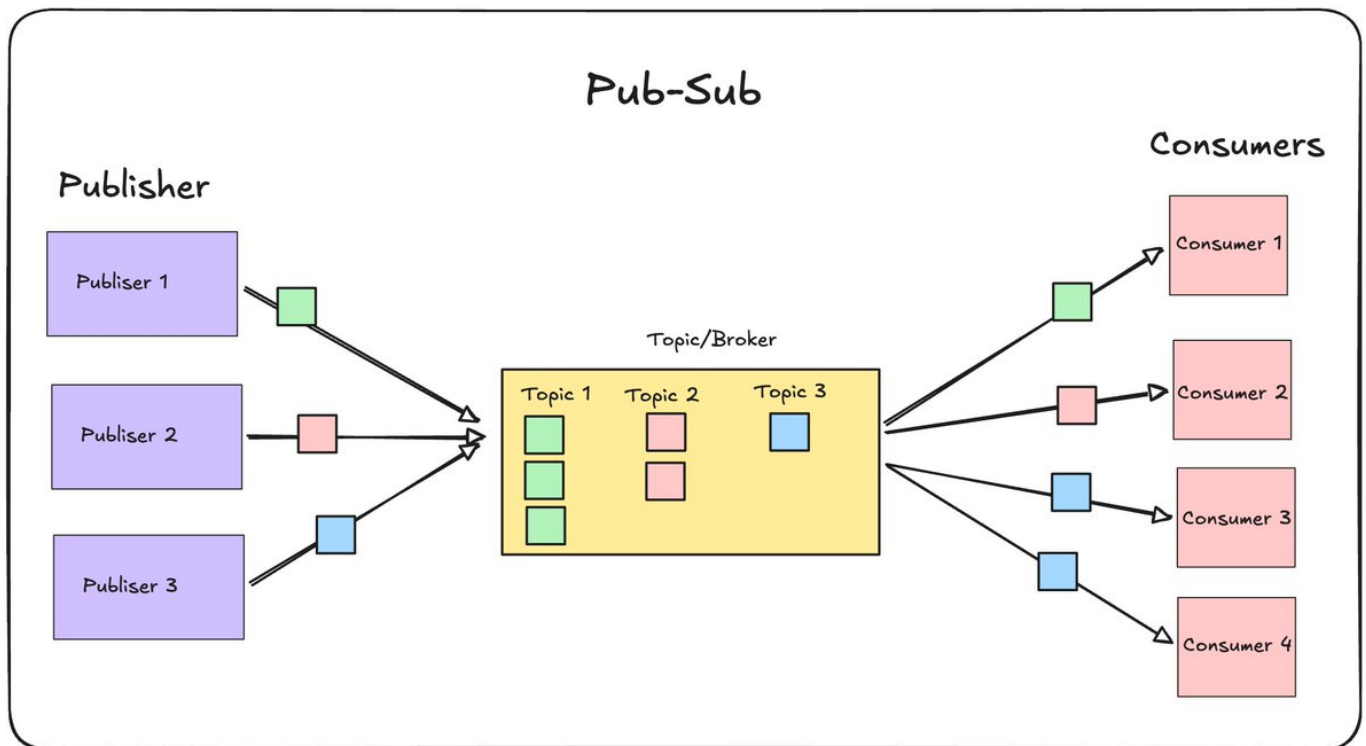
Producer moves on instantly. Consumer processes at its own pace.

Key properties

Decoupling: Producer and consumer don't need to know about each other

Buffering: Absorbs traffic spikes; consumer processes at its own speed

Reliability: Messages persist in the queue until successfully processed



Async by nature: Producer never waits for a response.

A Message Queue is an important component of Event-Driven Architecture.

25. Pub-Sub or Publish-Subscribe

Pub-Sub

A message queue has one producer and one consumer processing each message. Pub/Sub solves a different problem: what if multiple services need to react to the same event?

A publisher sends a message to a topic, and every subscriber listening to that topic gets a copy; unlike a queue where each message goes to one consumer, pub/sub fans out each message to all subscribers.

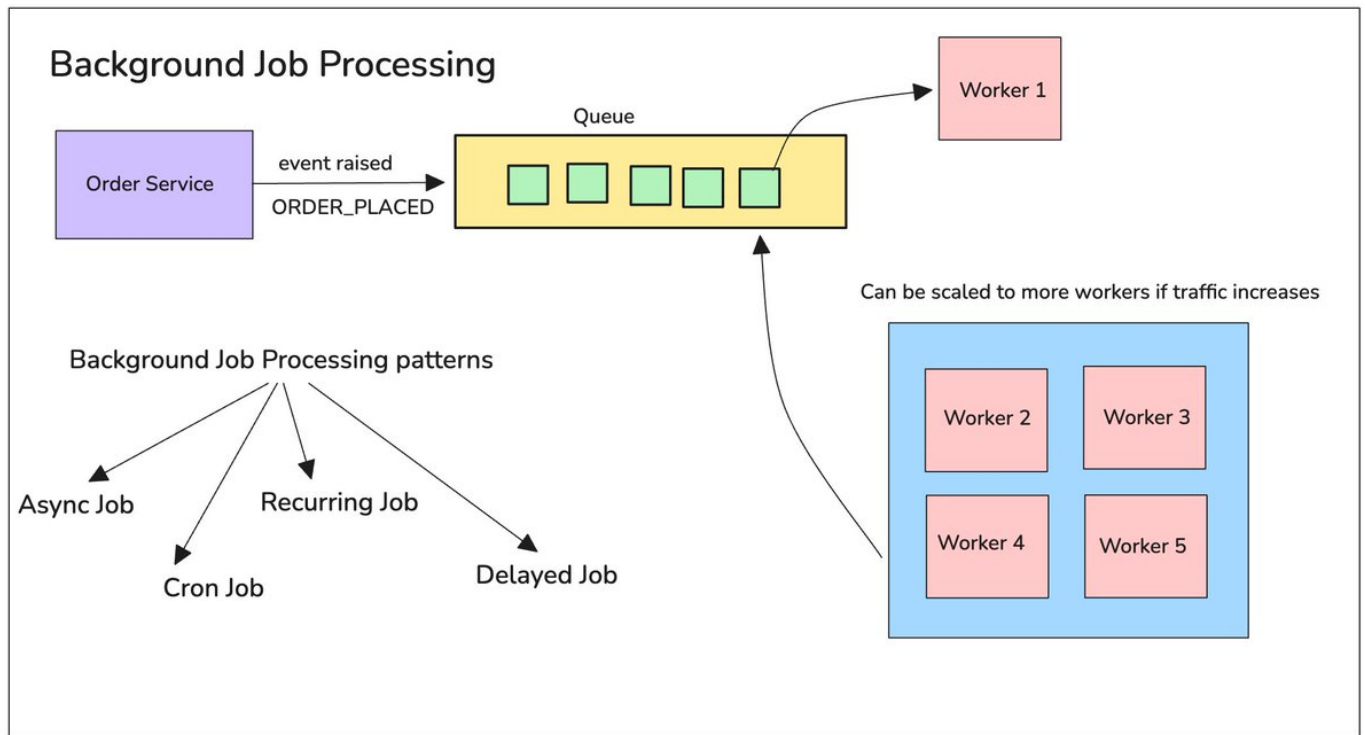
The publisher doesn't know or care who's subscribed. It just publishes.

Queue vs Pub/Sub:

Queue: one message, consumed by ONE consumer

Pub/Sub: one message, delivered to EVERY subscriber

26. Background Processing Job



Background Job Processing

Imagine a user places an order for a product. You are designing a feature where once the order is placed, the system should

Send the user a confirmation

Generate the order invoice PDF and send it by mail

Send an SMS to their number with the order number and estimated date of delivery

Sync the data with analytics

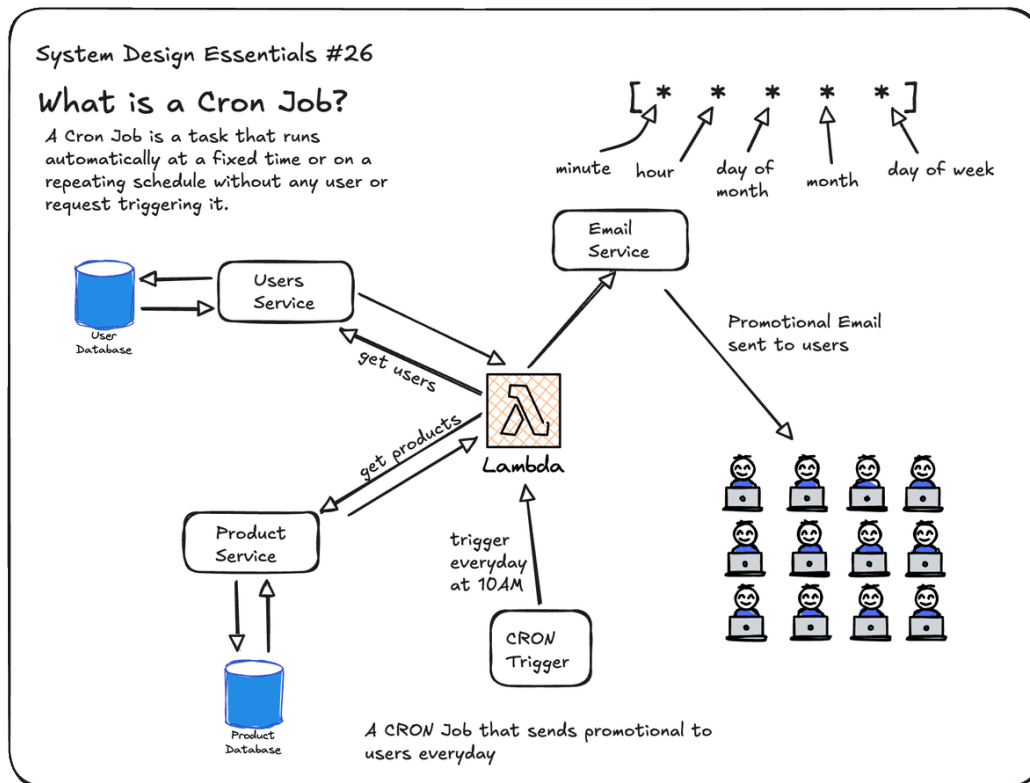
If we do all these tasks inside the same API call, i.e., the Create Order API, the user will have to wait for a longer time to receive the response from the service. In peak traffic times, the system might even take longer, or the request can even fail.

This is exactly the problem Background Job Processing solves.

Instead of doing everything in a synchronous manner, the idea is to move the non-critical jobs to the background to process later and finish the main request first. The main request is handled first, with all the critical operations performed, and the response is sent back. The non-critical operations are performed later.

Core Building Blocks:

Job/Task: a unit of work that needs to run (e.g., "send this email," "generate this report")



Job Queue: where jobs wait until a worker is free to process them (Redis, RabbitMQ, SQS, BullMQ)

Worker: a separate process that picks up jobs from the queue and executes them.

Background Processing Job Patterns:

Async or event-based

Scheduled or Cron Job

Recurring Jobs

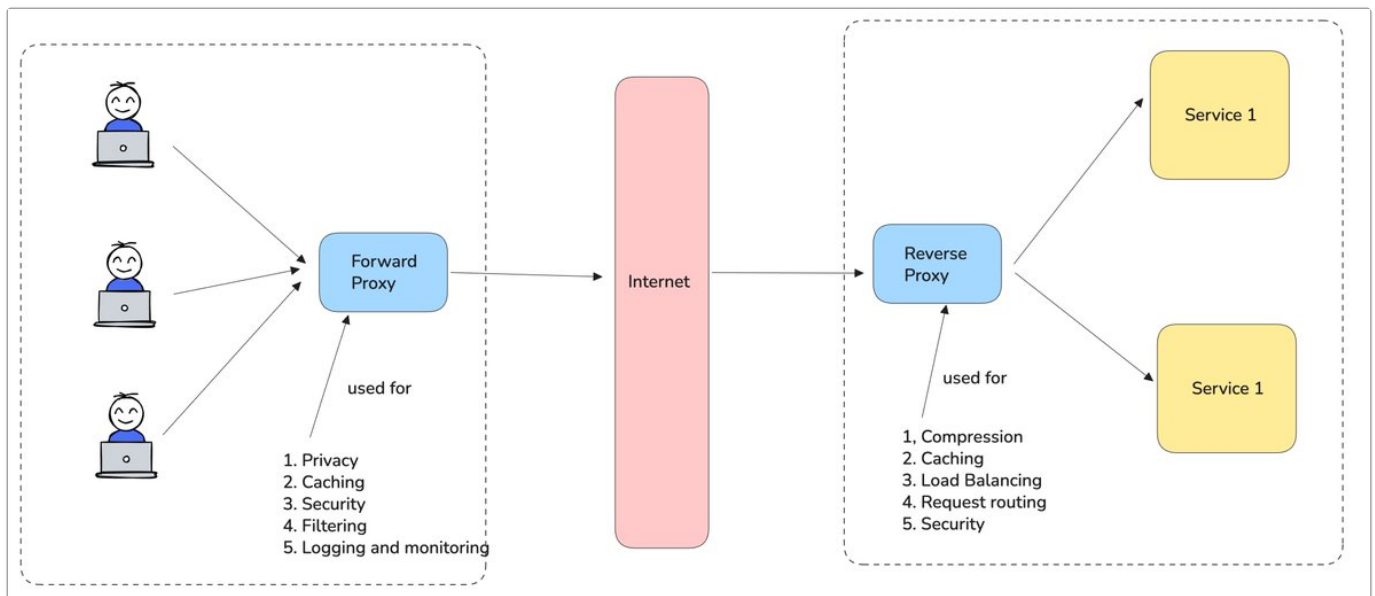
Delayed Jobs

27. Cron Job

Cron Job

A Cron Job is a task that runs automatically at a fixed time or on a repeating schedule without any user or request triggering it.

Regular Request: Client -> sends request -> server responds



Cron Job: Clock hits scheduled time -> job gets triggered

Cron jobs are scheduled using a specific syntax

[* * * * *]

0 0 * * * -> runs every day at midnight

0 */6 * * * -> runs every 6 hours

Use cases of Cron jobs:

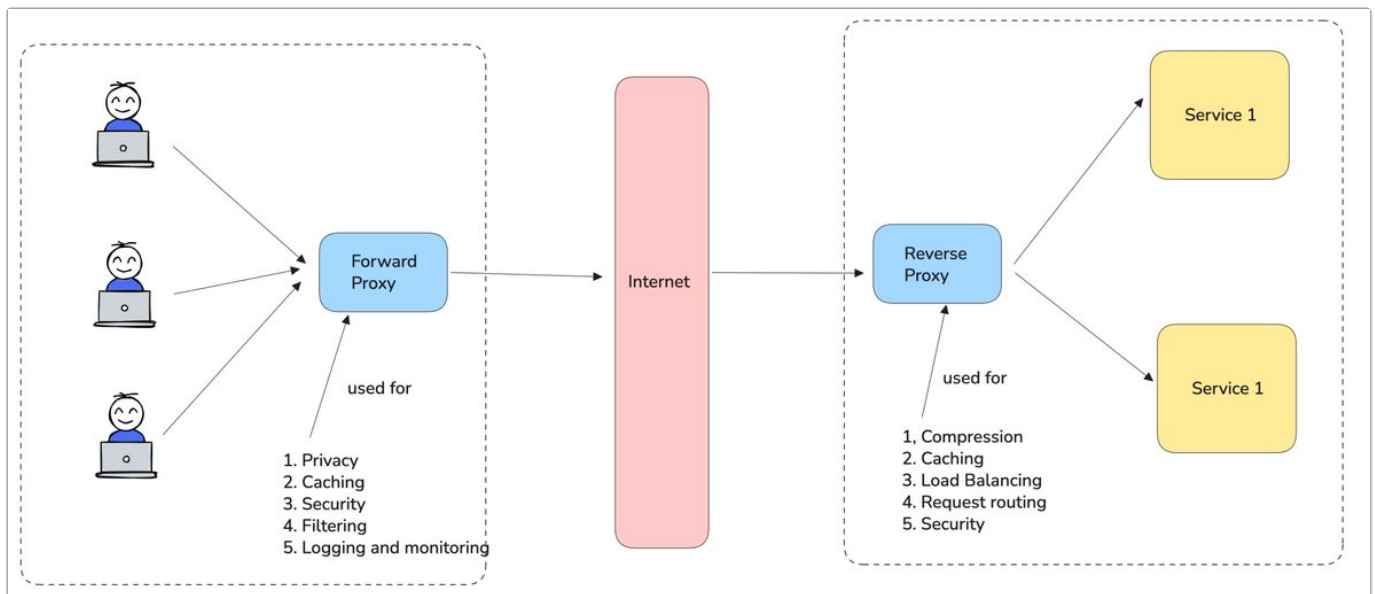
Daily Database Backup

Daily Report Generation

Clean up data every few hours

Send email alerts every 6 hours

28. Reverse Proxy



Reverse Proxy

A Reverse Proxy sits in front of the server, hiding the server from the client. Instead of clients talking directly to your actual server(s), they talk to the reverse proxy first. The proxy forwards the request to the right server internally, gets the response, and sends it back to the client. From the client's point of view, the proxy IS the server, and they never see what's actually behind it.

Without Reverse Proxy:

Client -> Server (client hits server directly)

With Reverse Proxy:

Client -> Reverse Proxy -> Server(s) (client only sees the proxy)

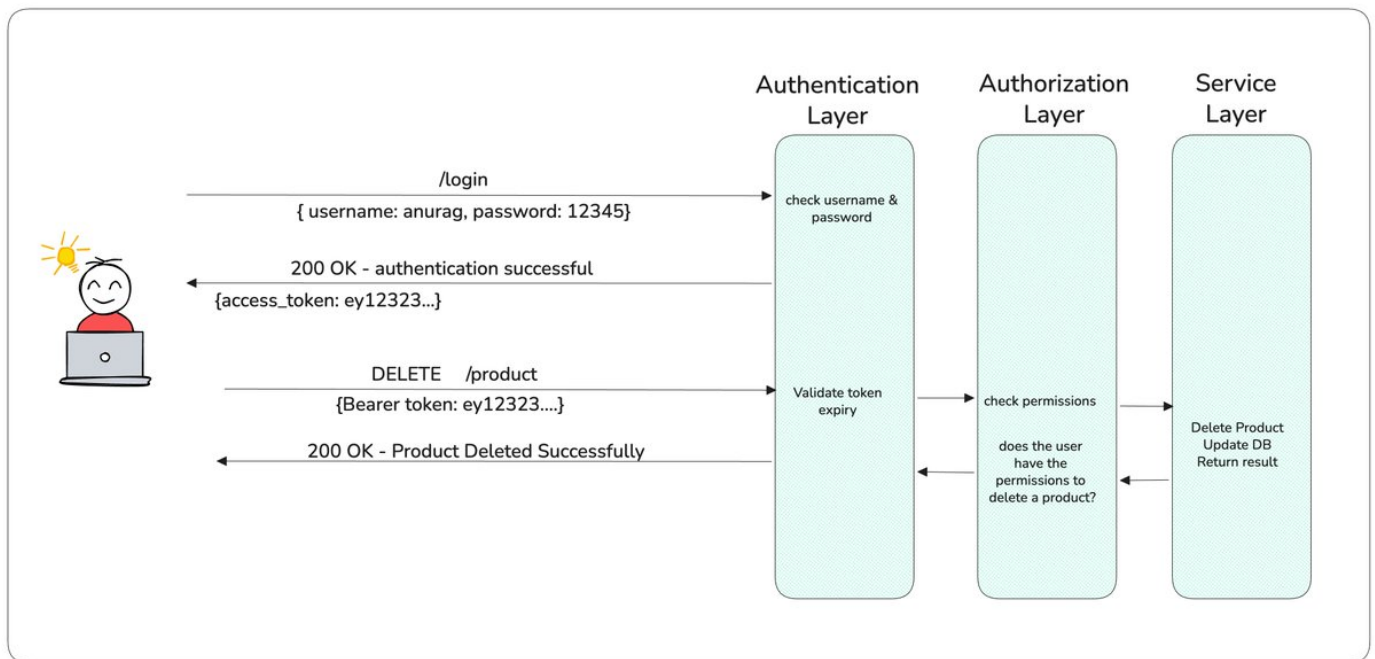
Use Cases of Reverse Proxy

Load Balancing

SSL Termination

Caching

Security



Request Routing

Compression

29. Forward Proxy

Forward Proxy

A Forward Proxy sits between a client and the internet and acts on behalf of the client. Instead of the client talking directly to the server, it talks to the proxy first. The proxy forwards the requests and accepts the response on the client's behalf. From the server's point of view, the request came from the proxy and not from the client. The server is not aware of the actual client.

Use Cases of Forward Proxy

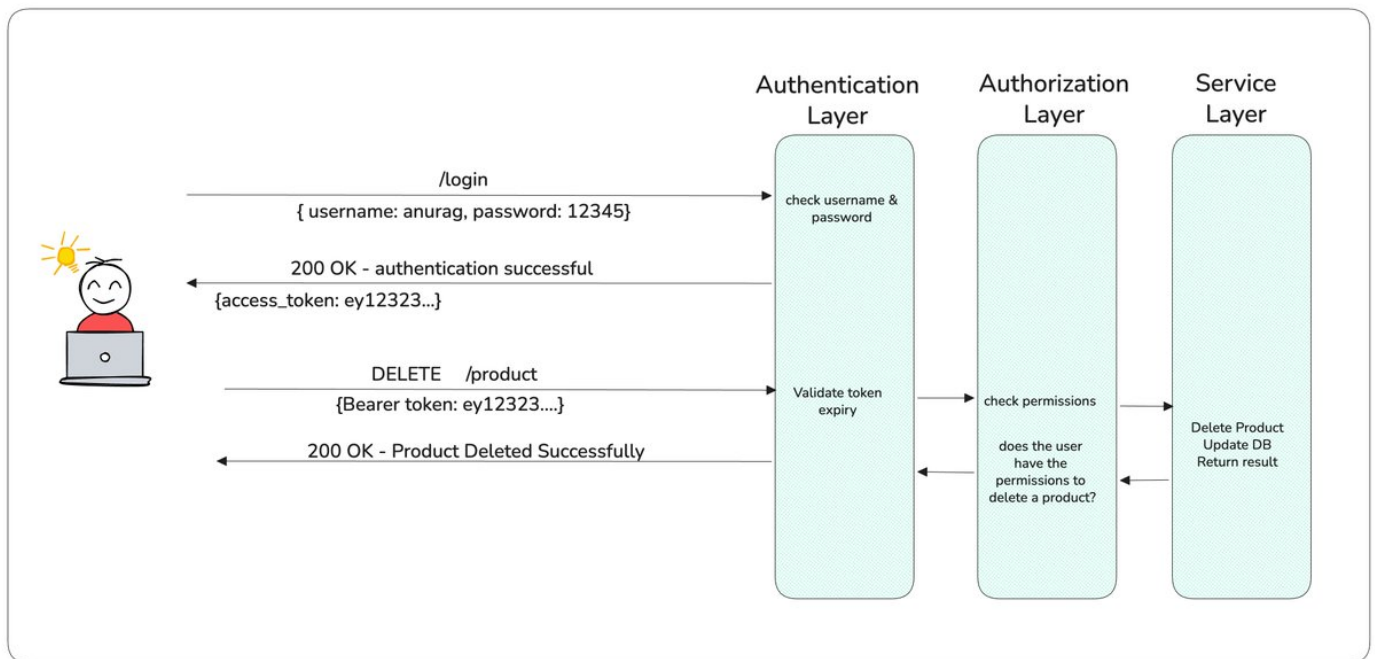
Privacy

Content filtering

Caching

Bypass geo-restrictions

Logging and Monitoring



Forward Proxy -> sits in front of the CLIENT, hides the client from the server. Reverse Proxy -> sits in front of the SERVER, hides the server from the client.

30. Authentication

Authentication

Authentication is the process of verifying a user's identity. Imagine an Employee Management application. For any user to use this application, they need to enter their username and password.

User -> Username + Password -> Server verifies credentials -> Identity confirmed -> User logged in

The server now knows who the user is. This is Authentication.

Common authentication methods:

Username + Password

OTP

Biometrics

API Keys

JSON Web Token - JWT

JWT is a token used for authentication and authorization purpose.

Header

Contains the algorithm used to sign the token, like HS256 or RS256.

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Payload

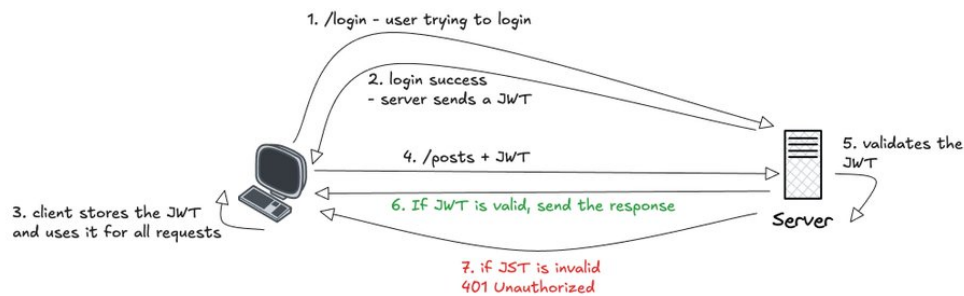
Contains the actual data, called claims. Things like userId, role, and expiry time.

```
{
  "userId": 102,
  "name": "Anurag",
  "role": "admin",
  "iat": 1732608000,
  "exp": 1732611600
}
```

Signature

Created by signing the header and payload with a secret key. This is what proves the token hasn't been altered.

```
HMACSHA256(
  base64UrlEncode(header)
  + "." + base64UrlEncode(payload),
  secretKey
)
```



OAuth

JWT

Session-based authentication

31. Authorization

Authorization

Authorization is the process of verifying if the user has enough permissions to act. A user can be authenticated but not authorised for a particular resource or action.

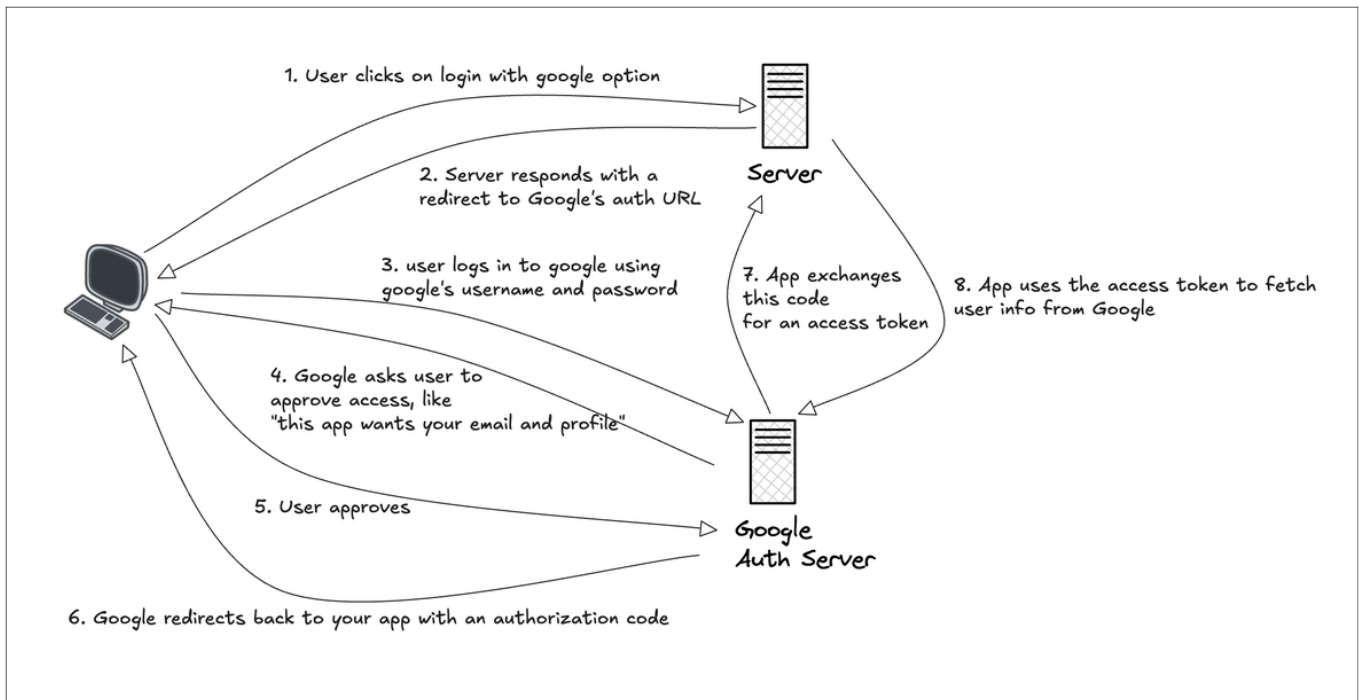
For example, an Employee Management application has two types of users

Admin

Add employees

Delete employees

Update employee details



View salary information

Employee

View own profile

Update own profile

View assigned tasks

Now imagine an employee tries to delete another employee:

The user is authenticated, but not authorised to perform that action.

Authentication vs Authorisation

Authentication = Who are you?

Authorisation = What are you allowed to do?

32. JWT - JSON Web Token

Parameters	Local Storage	Session Storage	Cookies
Scope	All tabs/windows	Single tab/window	Domain wide
Capacity	5-10mb	5-10mb	<u>4kb</u> per cookie
Expiry	Never	Tab closure	configurable
Data Sharing	Client side	Client side	Client and Server
Sent with requests	No	No	Yes
Compatibility	HTML 5	HTML 5	HTML 4/5

JSON Web Token

JWT, also called a JSON Web Token, is a token used for Authentication and authorisation. It lets a server verify a user without looking anything up in a database, since the token itself carries the data and a signature proving it wasn't tampered with.

A JWT has three components

Header

Contains the algorithm used to sign the token, like HS256 or RS256.

Payload

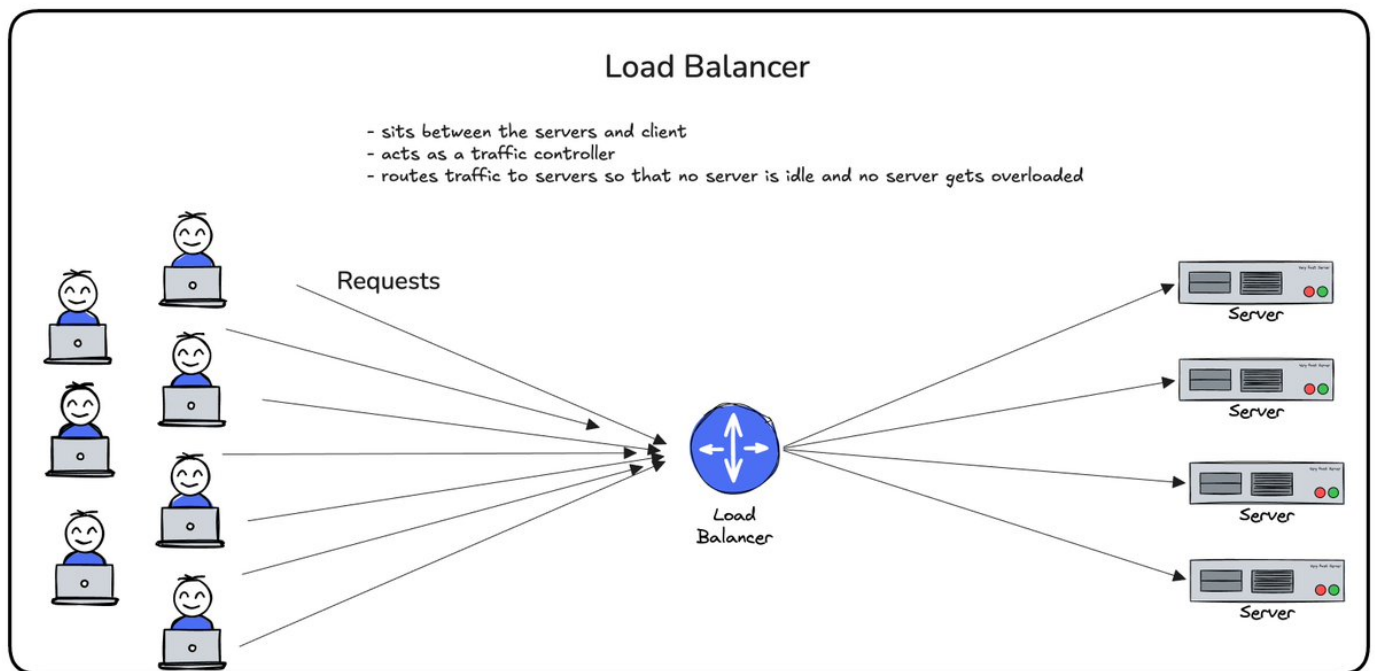
Contains the actual data, called claims. Things like userId, role, and expiry time.

Signature

Created by signing the header and payload with a secret key. This is what proves the token hasn't been altered.

How it works

Login -> Server creates a JWT, signs it with a secret -> sends it to client



Client -> stores the token, sends it in the Authorization header on every request

Server -> verifies the signature -> if valid, trusts the data inside, no database lookup needed.

33. OAuth

OAuth

OAuth is a way to let a user log in or grant access using an existing account, like Google or GitHub, without giving your app their password.

The problem it solves:

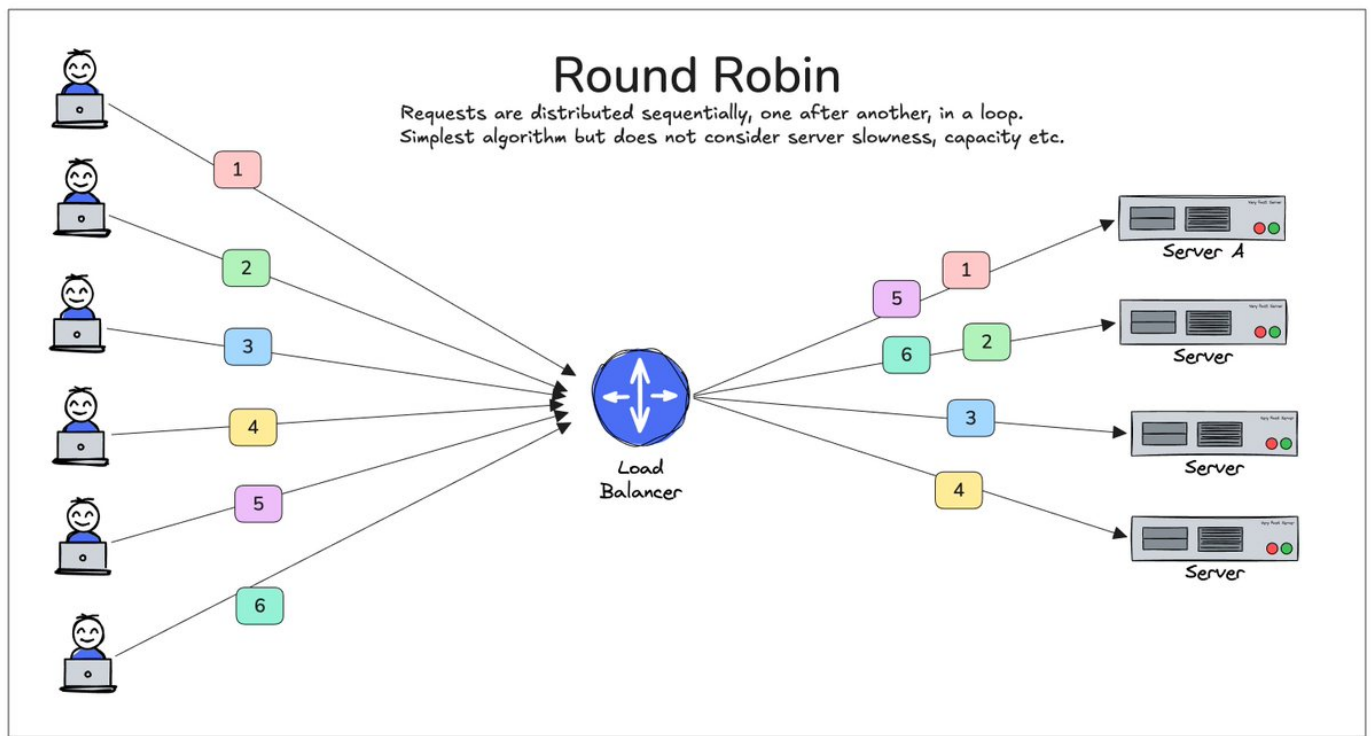
Without OAuth, if you want to let users log in with Google, you'd need Google to somehow share the user's password with your app. That's a terrible idea, since your app would now be able to log into their actual Google account.

OAuth avoids this entirely. Your app never sees the password.

What OAuth actually is:

An authorization framework where a user grants your app limited access to their data on another service, without sharing credentials. Your app gets a token instead, scoped to only what it needs.

OAuth flow:



User clicks Login with Google

App redirects user to Google's login page

User logs in directly on Google, enters password there, not on your app

Google asks the user to approve access, like "this app wants your email and profile"

User approves

Google redirects back to your app with an authorization code

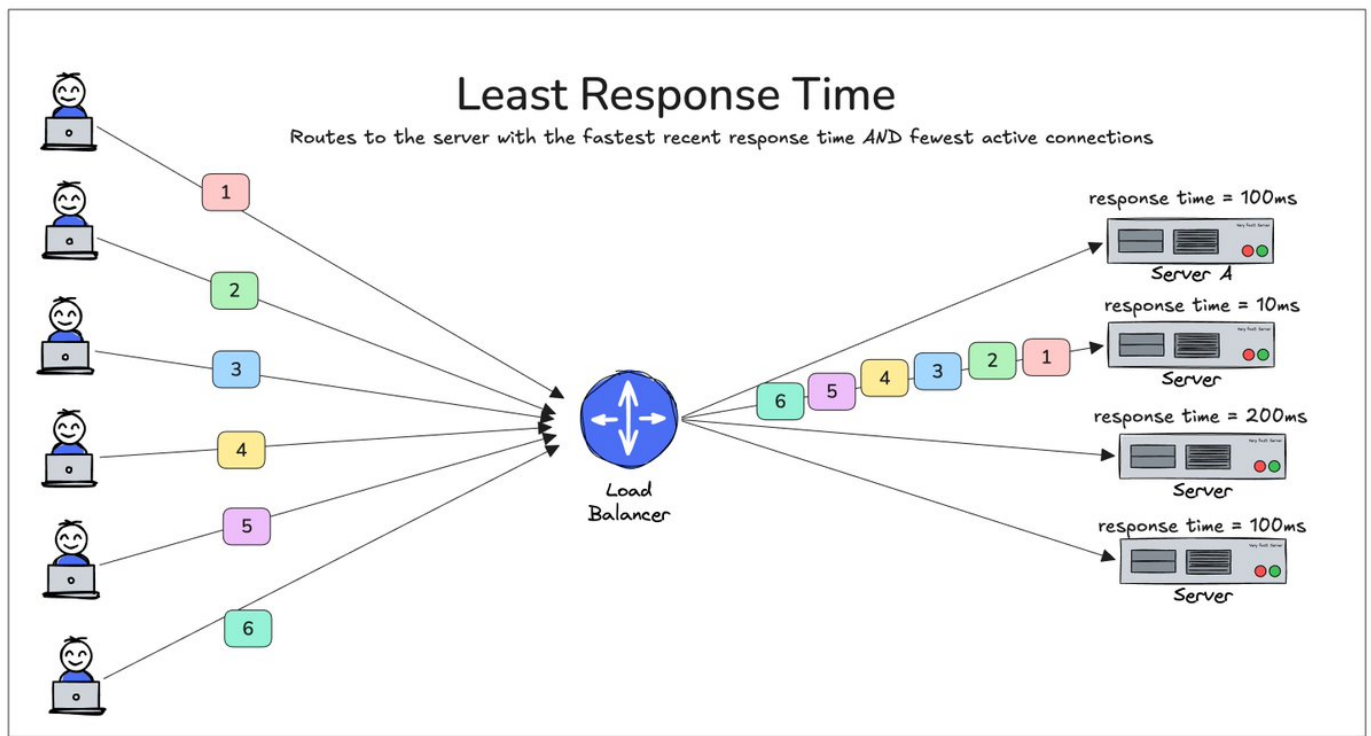
App exchanges this code for an access token

App uses the access token to fetch user info from Google

34. Cookies, Local Storage and Session Storage

Local Storage vs Session Storage vs Cookies

Client-side storage lets the browser hold small pieces of data, so the app doesn't have to fetch everything from the server every time. Here are the three common options.



Cookies

Small pieces of data, sent automatically with every HTTP request to the server.

Local Storage:

Stores data in the browser; persists even after the browser is closed and reopened.

Session Storage:

Same as Local Storage, but data only lasts for that browser tab's session.

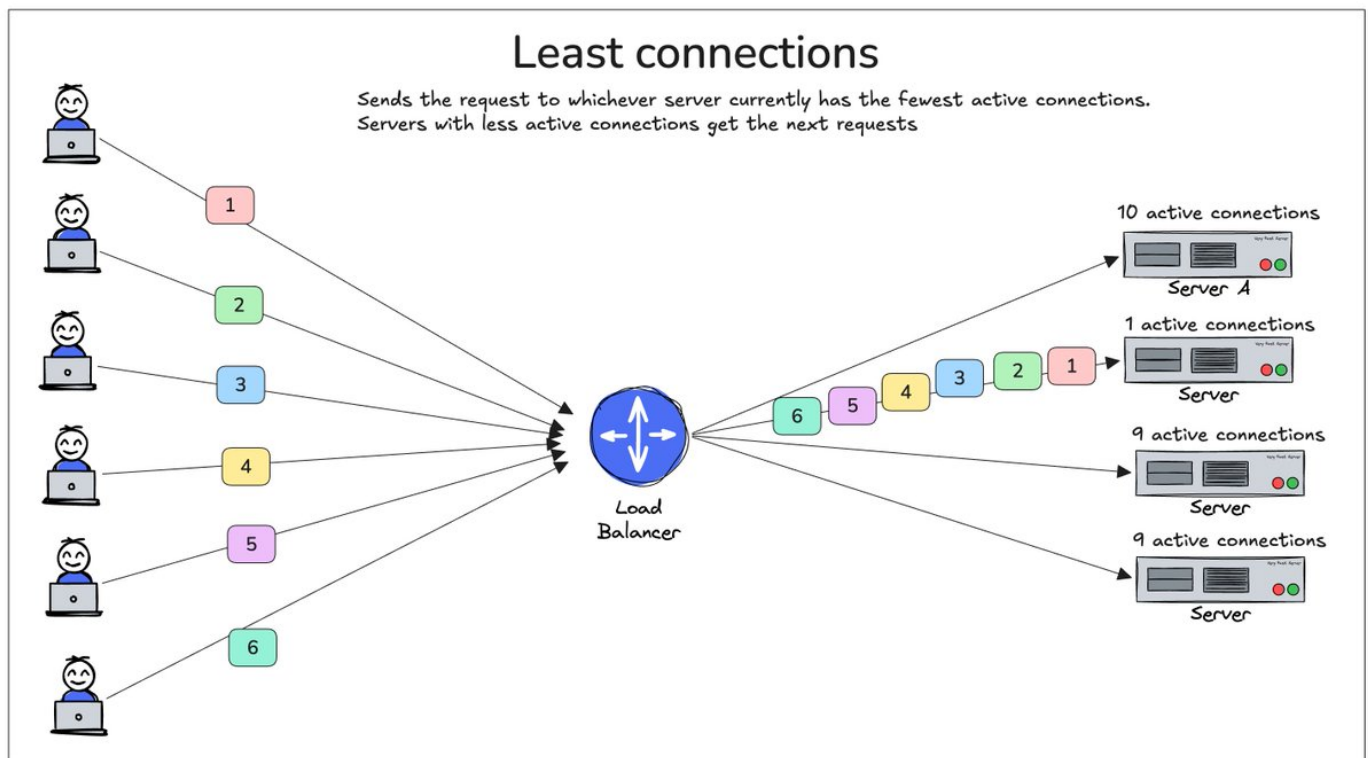
35. Load Balancing

Imagine you're building an E-Commerce application and hosting a big sale at midnight. Suddenly, thousands of users start visiting your application. Your single server receives more requests than it can handle. So how do we handle increasing traffic?

By adding more servers.

But now another question arises:

How do we decide which request goes to which server?



This is where Load Balancing comes in.

Load Balancing is the process of distributing incoming requests across multiple servers. The Load Balancer sits in front of your servers and decides where each incoming request should go so that one server does not get overloaded with requests and the rest of the servers don't sit idle. Where the request should go depends on the Load Balancing Algorithm.

36. Load Balancing Algorithm - Round Robin

Load Balancing Algorithm - Round Robin

Requests are distributed sequentially across servers. Once it reaches the last server, the cycle starts again.

Request 1 -> Server A

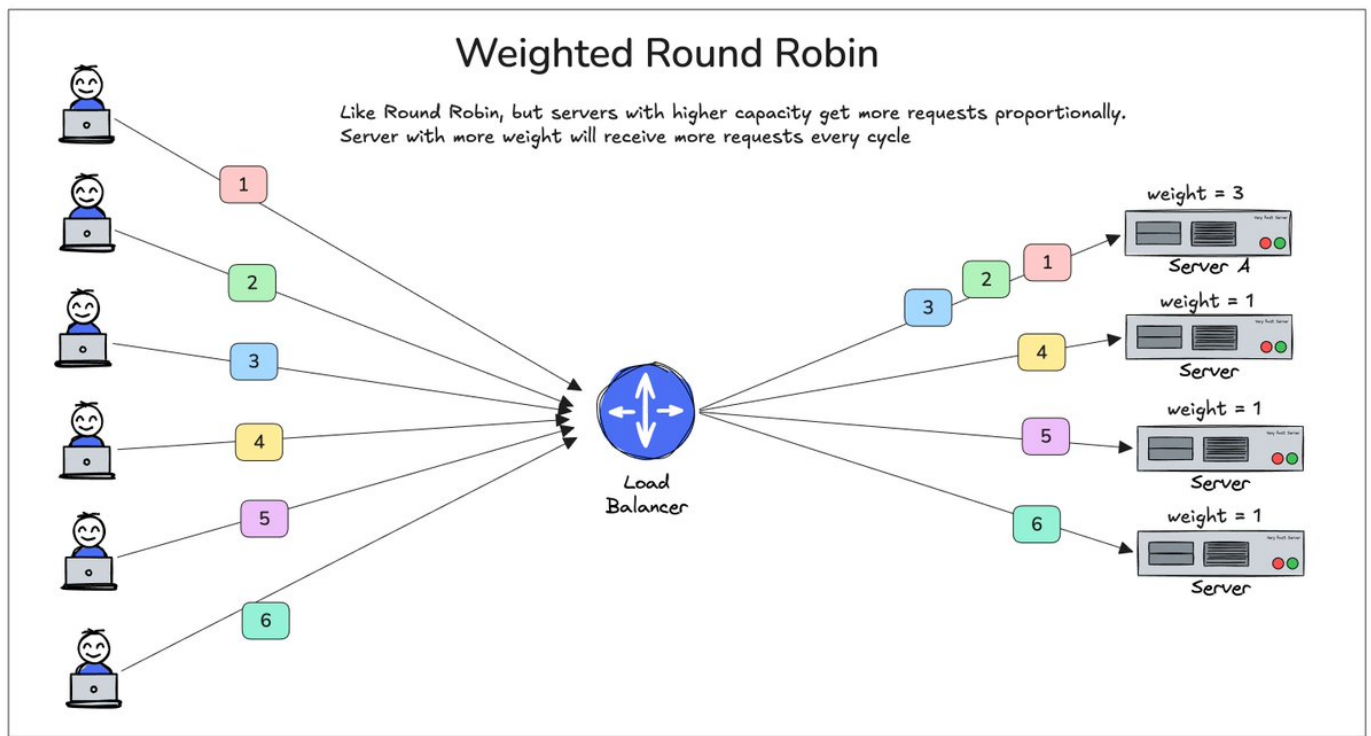
Request 2 -> Server B

Request 3 -> Server C

Request 4 -> Server D

Request 5 -> Server A (Requests start from Server A again)

Request 6 -> Server B



37. Load Balancing Algorithm - Least Response Time

Load Balancing Algorithm - Least Response Time

The request is sent to the server with the fewest active connections.

Server A -> 10 connections

Server B -> 3 connections <- Next request

Server C -> 9 connections

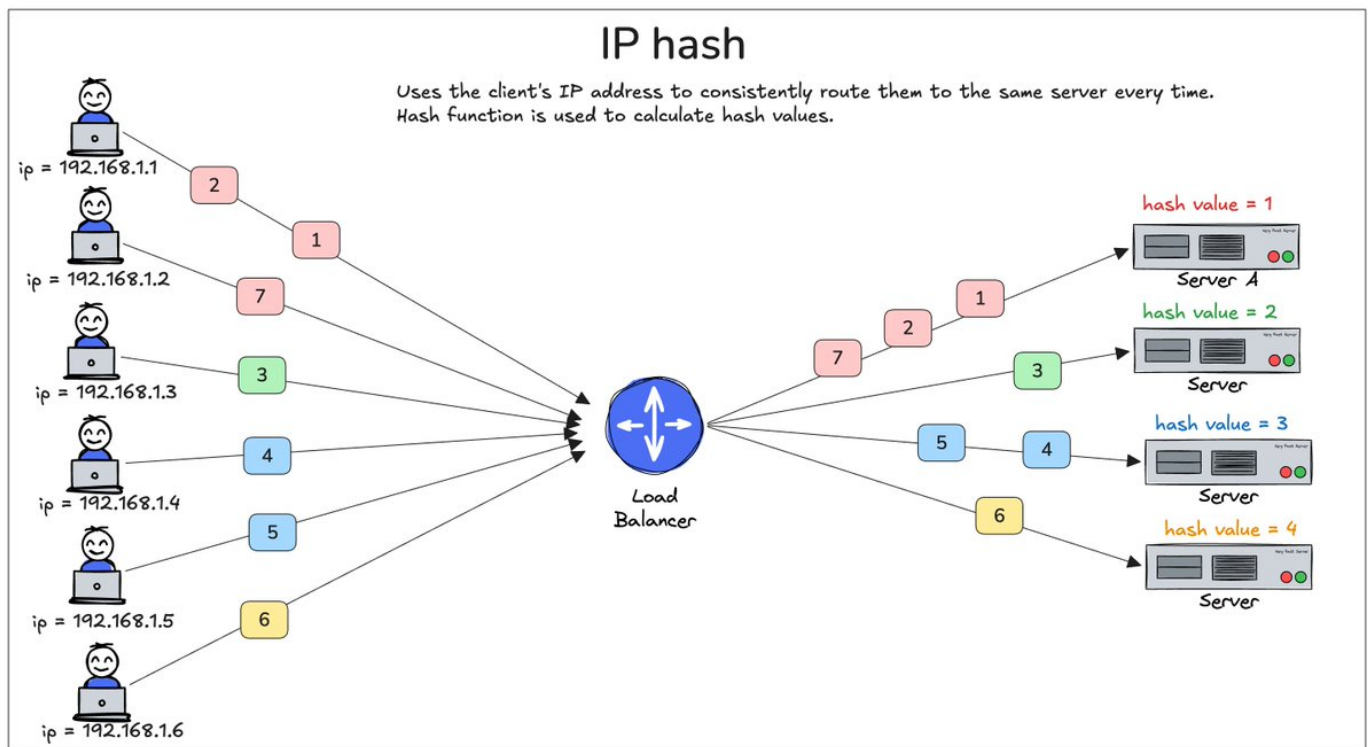
This can be better than Round Robin because not all requests take the same amount of time to complete.

38. Load Balancing Algorithm - Least Connections

Load Balancing Algorithm - Least Connections

Requests are routed based on server performance. The Load Balancer considers factors such as:

Current active connections



Recent response times

Requests are sent to servers that are responding faster and have less load.

39. Load Balancing Algorithm - Weighted Round Robin

Load Balancing Algorithm - Weighted Round Robin

Some servers may be more powerful than others. Weights allow stronger servers to receive more traffic.

Server A -> Weight 3

Server B -> Weight 1

For every cycle:

Server A -> 3 requests

Server B -> 1 request

40. Load Balancing Algorithm - IP Hash

Load Balancing Algorithm - IP Hash

The client's IP address is used to determine which server receives the request.

Client IP -> (Hash Function) -> Server B.

The same client will usually be routed to the same server. This can be useful for session persistence (sticky sessions).

That's it for the second part. Stay tuned for the third part, where I will explain topics like Caching and Caching algorithms, Rate Limiting Algorithms, Circuit Breakers, deployment strategies, etc.